

Semantic Web & Ontology



Rathachai CHAWUTHAI, PhD.

Department of Computer Engineering, School of Engineering
King Mongkut's Institute of Technology Ladkrabang

Rathachai Chawuthai



Rathachai



rathachai



rathachai.c

■ Now

- Assoc. Prof. at Computer Engineering
- King Mongkut's Institute of Technology Ladkrabang (KMITL)

■ Education Background

- B.Eng. at KMITL
- M.Eng. at AIT
- Ph.D at SOKENDAI, Japan

■ Work Experience

- Thomson Reuters Thailand (4 years)
- Punsarn Asia (1 years)
- NII, Japan (3 years)

Publications about Semantic Web

Journal Papers and Book Chapters

- Chawuthai, R., Takeda, H., Wuwongse, V., & Jinbo, U. (2018). Presenting and preserving the change in taxonomic knowledge for linked data. In *Semantic Web Journal*.
- Chawuthai, R., Jinbo, U., Takeda, H. (2018) "Linked Open Data Model for Taxonomic Information". In a book titled "Application of Semantic Technology in Biodiversity Science". IOS Press.

Conference and Workshop Papers

- Chawuthai, R., Kertkeidkachorn, N., & Racharak, T. (2023). Modelling an RDF Knowledge Graph with Transitivity and Symmetry for Bus Route Path Finding. In *Proceedings of the 2023 6th Artificial Intelligence and Cloud Computing Conference* (pp. 126-134).
- Chawuthai, R., Takeda, H., Wuwongse, V., & Jinbo, U. (2018). Presenting and preserving the change in taxonomic knowledge for linked data. In *Companion Proceedings of the The Web Conference 2018* (pp. 461-465).
- Chawuthai, R., & Takeda, H. (2016). Rdf graph visualization by interpreting linked data as knowledge. In *Semantic Technology: 5th Joint International Conference, JIST 2015, Yichang, China, November 11-13, 2015, Revised Selected Papers 5* (pp. 23-39). Springer International Publishing.
- Cazabet, R., Chawuthai, R., & Takeda, H. (2015). Using multiple-criteria methods to evaluate community partitions. *arXiv preprint arXiv:1502.05149*.
- Chawuthai, R., & Takeda, H. (2015). rSim: Simplifying an RDF Graph at the Visualization Tier for Non-Expert Users. In *ISWC (Posters & Demos)*.
- Chawuthai, R., Takeda, H., & Hosoya, T. (2015). Link prediction in linked data of interspecies interactions using hybrid recommendation approach. In *Semantic Technology: 4th Joint International Conference, JIST 2014, Chiang Mai, Thailand, November 9-11, 2014. Revised Selected Papers 4* (pp. 113-128). Springer International Publishing.
- Nonthakarn, C., Chawuthai, R., & Wuwongse, V. (2014). An Application Profile for Linked Teacher Profiles and Teaching Resources. In *The Emergence of Digital Libraries—Research and Practices: 16th International Conference on Asia-Pacific Digital Libraries, ICADL 2014, Chiang Mai, Thailand, November 5-7, 2014. Proceedings 16* (pp. 138-148). Springer International Publishing.
- Chawuthai, R., Takeda, H., Wuwongse, V., & Jinbo, U. (2013). A logical model for taxonomic concepts for expanding knowledge using Linked Open Data. *Semantics for Biodiversity (S4BioDiv 2013)*, 9.
- Chawuthai, R., Wuwongse, V., & Takeda, H. (2012). A formal approach to the modelling of digital archives. In *The Outreach of Digital Libraries: A Globalized Resource Network: 14th International Conference on Asia-Pacific Digital Libraries, ICADL 2012, Taipei, Taiwan, November 12-15, 2012. Proceedings 14* (pp. 179-188). Springer Berlin Heidelberg.

Agenda

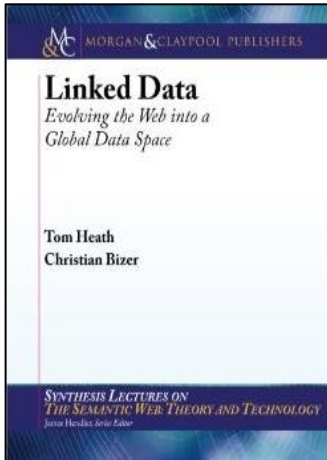
- Introduction
- RDF Graph
- RDF Schema
- Upper Ontology
- Designing Ontology
- Reasoning
- Query
- LOD





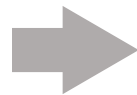
Introduction

References

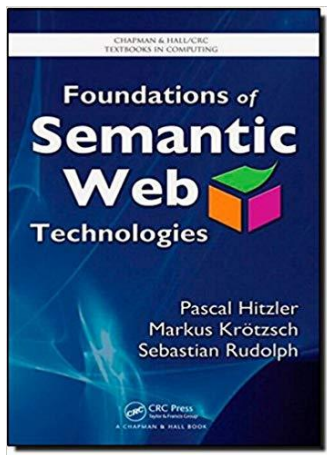


Linked Data: Evolving the Web into a Global Data Space

By Tom Heath , Christian Bizer



<http://linkeddatabook.com>



Foundations of Semantic Web Technologies

by Pascal Hitzler , Markus Krötzsch , Sebastian Rudolph



http://www.semantic-web-book.org/page/Foundations_of_Semantic_Web_Technologies

Acknowledgements



Prof. Vilas WUWONGSE



Prof. Hideaki TAKEDA

Lessons from AI (Artificial Intelligence)

AI Systems

Rule-Driven

expert systems

Data-Driven

machine learning systems

We need **DATA**

We need **Structured DATA**

We need **Meaningful & Structured DATA**

We need **Meaningful , Exchangeable & Structured DATA**

Open Data



Concordat on Open Research Data

The Concordat on Open Research Data has been developed by a UK multi-stakeholder group. This concordat will help to ensure that the research data gathered and generated by members of the UK research community is made openly available for use by others wherever possible in a manner consistent with relevant legal, ethical, disciplinary and regulatory frameworks and norms, and with due regard to the costs involved.

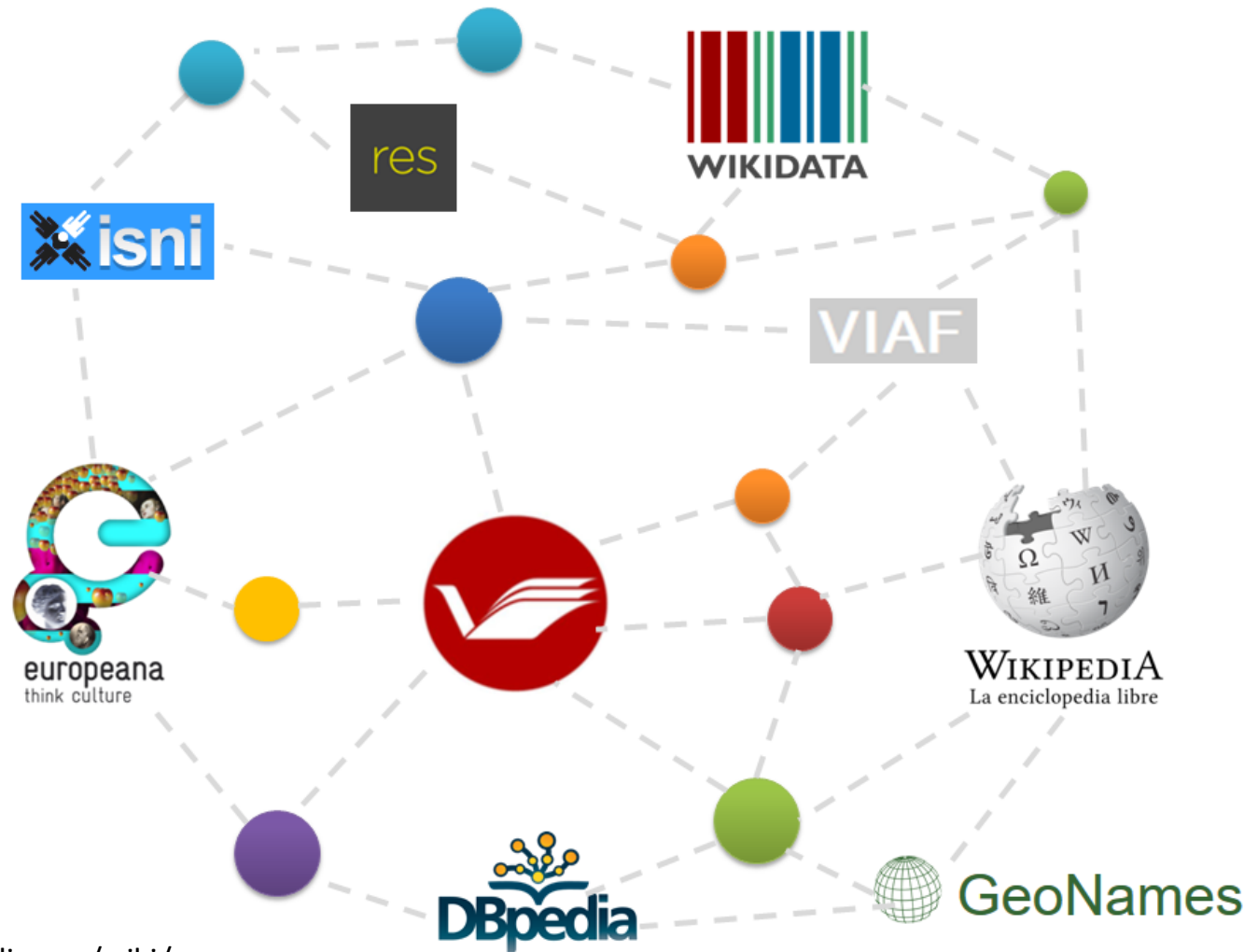


USA

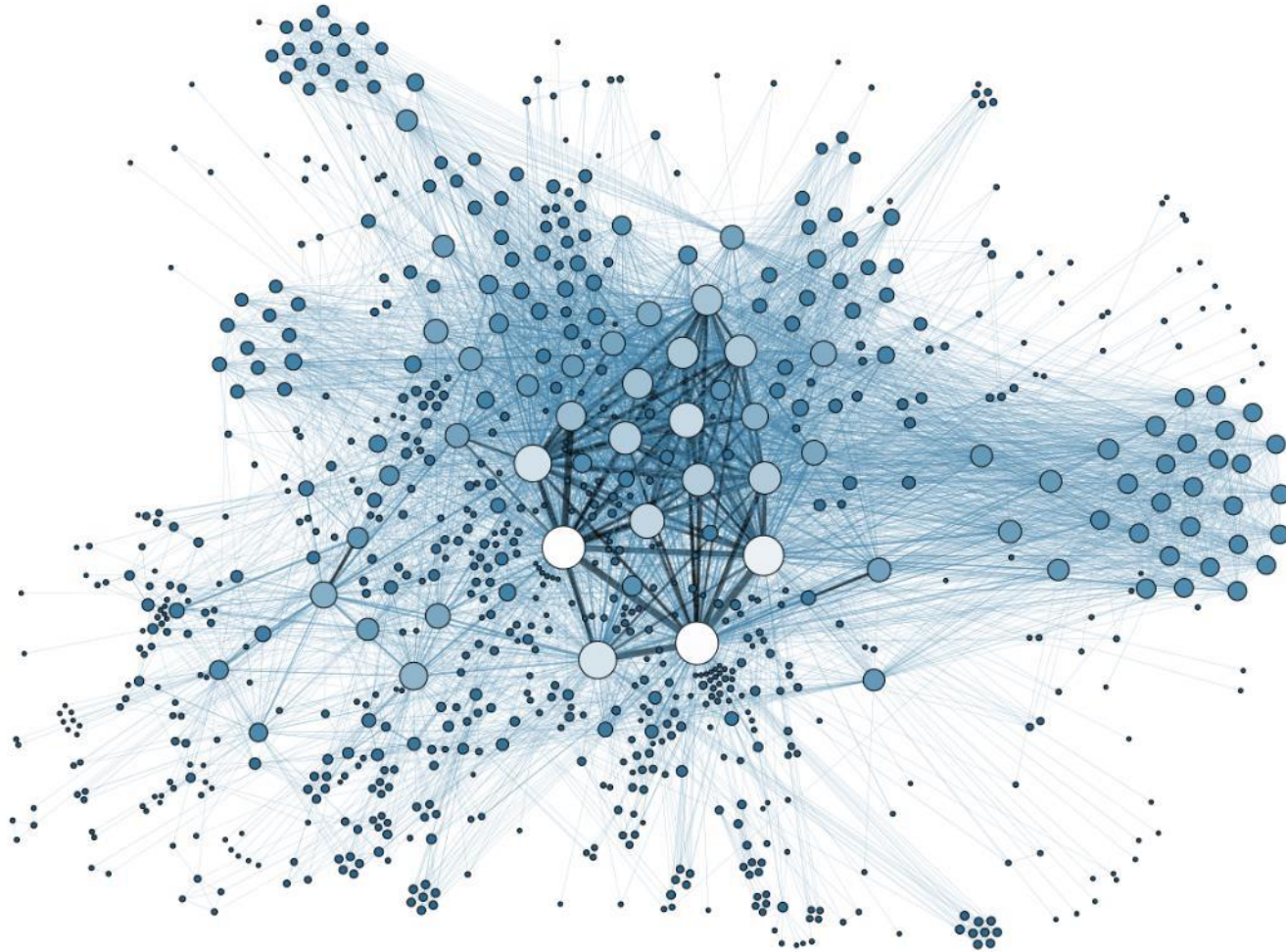
- USA: NSF DATA SHARING POLICY

Investigators are expected to **share with other researchers the primary data**, samples, physical collections and other supporting materials created or gathered in the course of work under NSF grants.

Linked Data



Linked Data



<http://sites.linkeddata.center/blog/archive/1-2-3linkedopendataarehere>

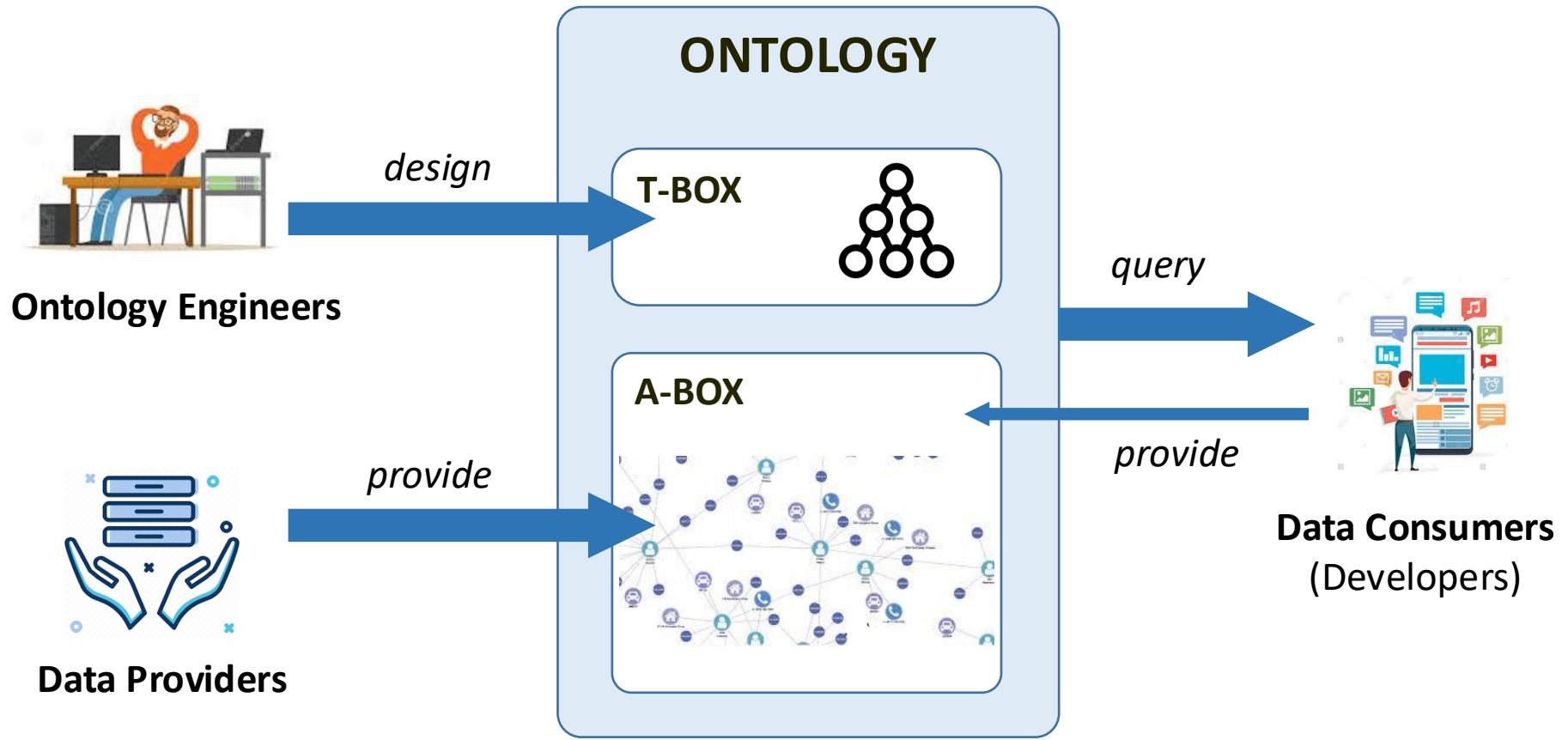
Linked Open Data

- Linked Data is about using the Web to connect related data that wasn't previously linked, or using the Web to lower the barriers to linking data currently linked using other methods.
- Global Knowledge Management System
- More specifically, Wikipedia defines Linked Data as "a term used to describe a recommended best practice for exposing, sharing, and connecting pieces of data, information, and knowledge on the Semantic Web using URIs and RDF."
- <http://linkeddata.org/>

Introduction

- **Linked Open Data** (LOD) is one practice approach of **Semantic Web**.
- One significant ability is to **integrate open data** from different schemas through the Internet.
- A **graph** is a data structure that is used in LOD.
- The **reasoning on graph** data together with schemas and ontologies can **improve the connection and accessibility** of data.
- Thus, we would like to provide background knowledge of **RDF graph, RDF Schema, ontology, reasoning, query, and LOD cloud**.

Roles in LOD



Roles and Skills



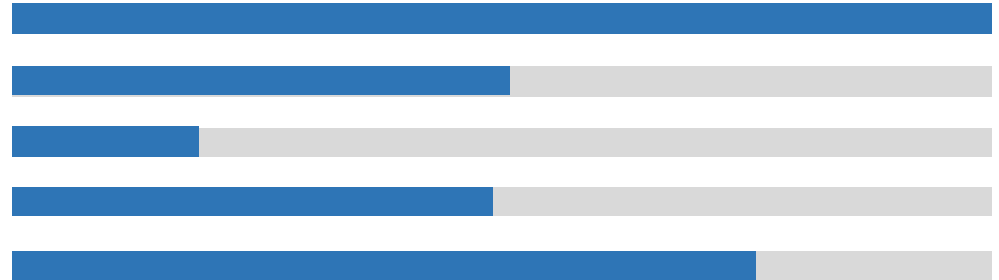
Ontology Engineers

RDF
RDFS & OWL
Reasoning
SPARQL
LOD



Data Providers

RDF
RDFS & OWL
Reasoning
SPARQL
LOD



Data Consumers (developers)

RDF
RDFS & OWL
Reasoning
SPARQL
LOD

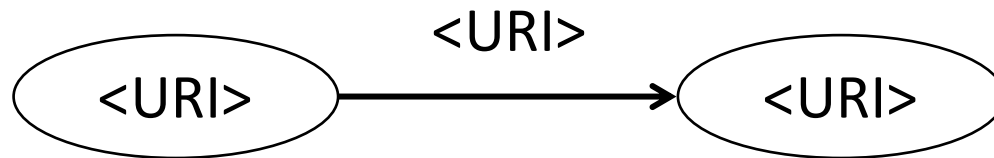




RDF Graph

RDF

- Resource Description Framework (RDF)
- Graph data model in LOD is presented by a direct graph. Nodes can be either named resources or literals, while links are named properties. Every resource and property is written using a **Uniform Resource Identifier (URI)**, and the structure of graph is modeled by **Resource Description Framework (RDF)**



URI

- Uniform Resource Identifier (URI)
- Every resource and property must be identified. In the RDF model, the **URI is used to be the identifier of resources and properties.**

- For example,

```
http://www.example.org/universities/Sokendai
```

```
http://www.example.org/students#Rathachai
```

```
http://www.example.org/terms#studiesAt
```

- are resources describing the terms “Sokendai”, “Rathachai”, and “studies at” respectively. URIs are used **to identified real-world objects and abstract concepts.**

Prefix

- LOD recommended that the URI should be **HTTP URI**, and clients can access each URI using the HTTP protocol and then get a returned document according to requested format.
- RDF allows having short-hand writing, for example if a prefix is defined as

```
@prefix unv: <http://www.example.org/universities/> .  
@prefix std: <http://www.example.org/students#> .  
@prefix : <http://www.example.org/terms#> .
```

- The former URIs can be shortened to be
unv:Sokendai, **std**:Rathachai, and **:**studiesAt.

Literal

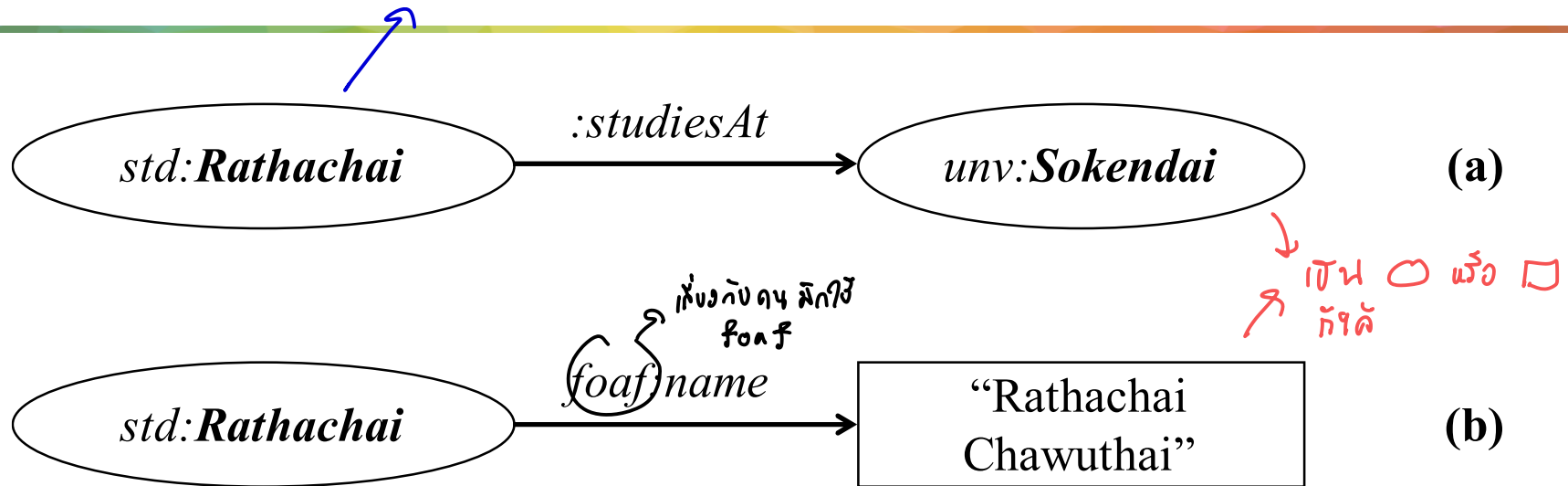
- Literals are texts that are typed and untyped. The untyped literal can be **any string** such as *“Rathachai Chawuthai”*, *“December 1986”*, *“18.12”*, and *“+8180-7999-1818”*.
- A string can be ended with a **language tag** in order to inform the language of a text. In addition, the **typed literal** contain a property value together with a URI of datatype.
- For example
 - `"Linked Open Data"@en` presents that this string is in English.
 - `"555"^^xsd:integer` presents an integer of 555.
 - `"1986-12-18"^^xsd:date` presents a date of Dec 18th, 1986.

Triple *s p o*

- Triple is a binary **relation between two entities**, and it is the fundamental unit of an RDF graph. It includes the sequence of a **subject**, a **predicate**, and an **object**. Subjects and predicates are resources represented by URIs, but objects can be either resources or literals.
- The formal representation of triple is $\langle s, p, o \rangle$ where **s** is a subject, **p** is a predicate or a property, and **o** is an object. In the graph diagram, the **URI of a resource is commonly written inside an eclipse**, a literal is presented by a quoted text inside a rectangle, and an arrows with the **URI of predicate** are drawn from a subject to an object.

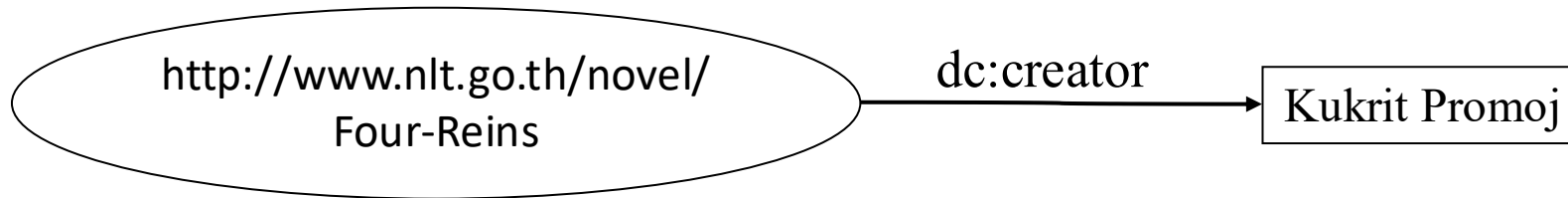
Triple

Predicate เป็น URI ที่นำด้วย 95



- The resource *std:Rathachai* is a subject, the *:studiesAt* is a predicate, and the *unv:Sokendai* is an object; and humans can simply interpret that *Rathachai studies at Sokendai*.
- The resource *std:Rathachai* is a subject, the *foaf:name* is a predicate, and the "Rathachai Chawuthai" is an object literal, and humans can simply interpret that *Rathachai's name is "Rathachai Chawuthai"*.

RDF Syntaxes and Dialects



```
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:dc="http://purl.org/dc/elements/1.1/">
  <rdf:Description rdf:about="http://www.nlt.go.th/novel/Four-Reins">
    <dc:creator>Kukrit Pramoj</dc:creator>
  </rdf:Description>
</rdf:RDF>
```

RDF/XML

↗ Predicate

```
<http://www.nlt.go.th/novel/Four-Reins>
<http://purl.org/dc/elements/1.1/creator> "Kukrit Pramoj" .
```

N-Triples

```
@prefix dc: <http://purl.org/dc/elements/1.1/> .
```

```
<http://www.nlt.go.th/novel/Four-Reins> dc:creator "Kukrit Pramote" .
```

RDF/Turtle

RDF Expression

- A triple can be expressed by several formats such as Turtle, XML, JSON-LD, and N-Triples, etc. In this thesis, we commonly use the **Turtle** (or RDF/Turtle) because it is **convenient to read and write by humans**.
- The Fig (a) can be expressed in Turtle format by



```
<http://www.example.org/students#Rathachai>  
  <http://www.example.org/terms/studiesAt>  
    <http://www.example.org/universities/Sokendai> .
```

Turtle

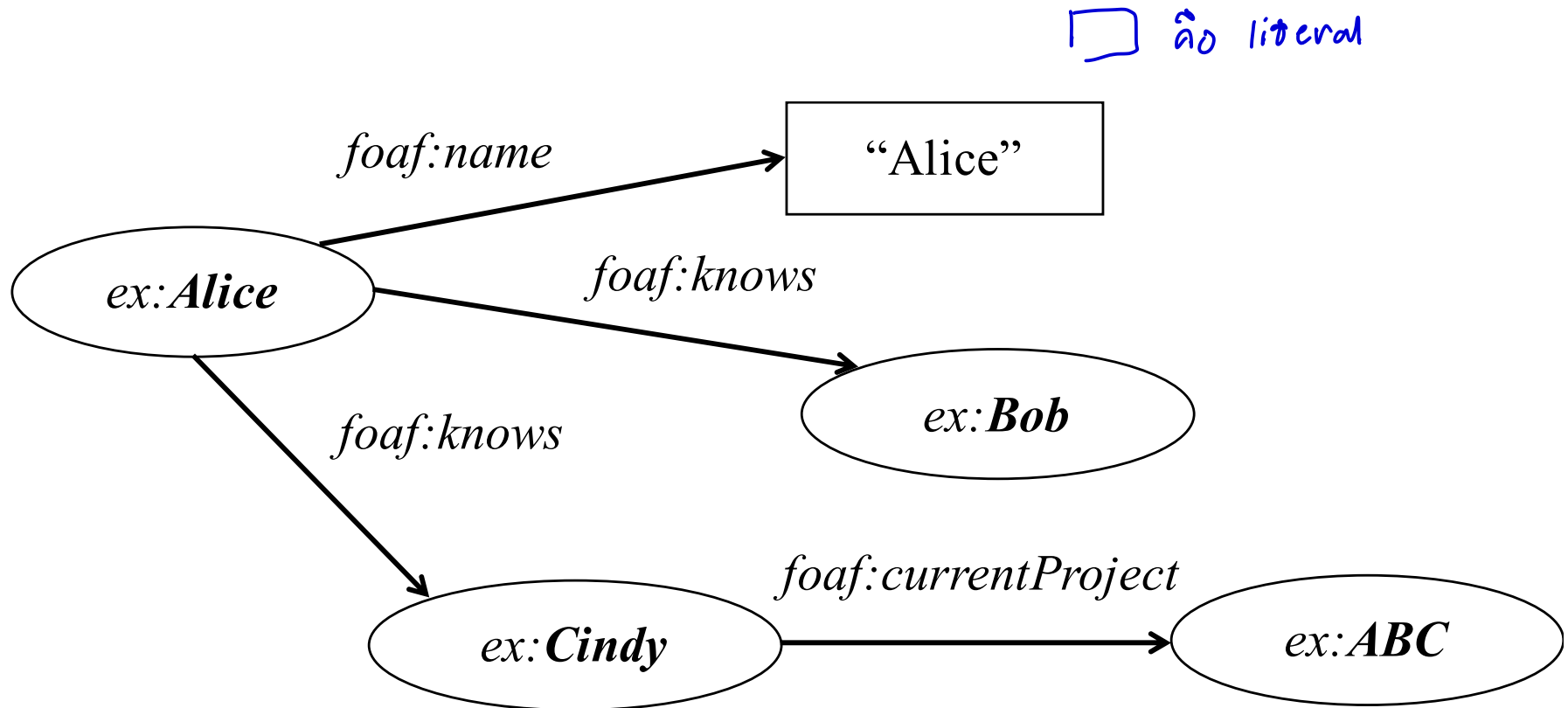
- Fig (a) can be shortened into

```
@prefix unv: <http://www.example.org/universities/> .  
@prefix std: <http://www.example.org/students#> .  
@prefix :    <http://www.example.org/terms/> .  
  
std:Rathachai    :studiesAt    unv:Sokendai .
```

- Fig (b) can also be shorten into

```
@prefix std: <http://www.example.org/students#> .  
@prefix foaf: <http://xmlns.com/foaf/0.1/> .  
  
std:Rathachai    foaf:name "Rathachai Chawuthai" .
```

RDF Graph



RDF Graph

- RDF graph that can be called as RDF model is a set of triples. It can be written in Turtle, so triples can be stated in curly brackets such as *{ :s1 :p1 :o1 . :s2 :p2 :o2 . }*, and the expression can be abbreviated as the following list.
- It is noted that the declaration of **prefixes** is sometime avoid in some expressions in order to make it be **simpler and easier for reading** and **reduce text length**.

RDF Graph

- If triples share the same subject and predicate, it can put objects in an object list and separate them by comma (,).
- For example, a graph

`{ :s1 :p1 :o1 . :s1 :p1 :o2 . }`

can be abbreviated as

`{ :s1 :p1 :o1 , :o2 . }`

Turtle 46
อ๋อ! งั้นใช่แล้ว

RDF Graph

- If triples share the same subject, it can put predicates and objects in a predicate list and separate them by semi-colon (;).
- For example, a graph

`{ :s1 :p1 :o1 . :s1 :p2 :o2 . }`

can be abbreviated as

`{ :s1 :p1 :o1 ; :p2 :o2 . }`

RDF Graph

- If triples contain the chain of same blank node, it can put the relative cause in square brackets.
- For example, a graph

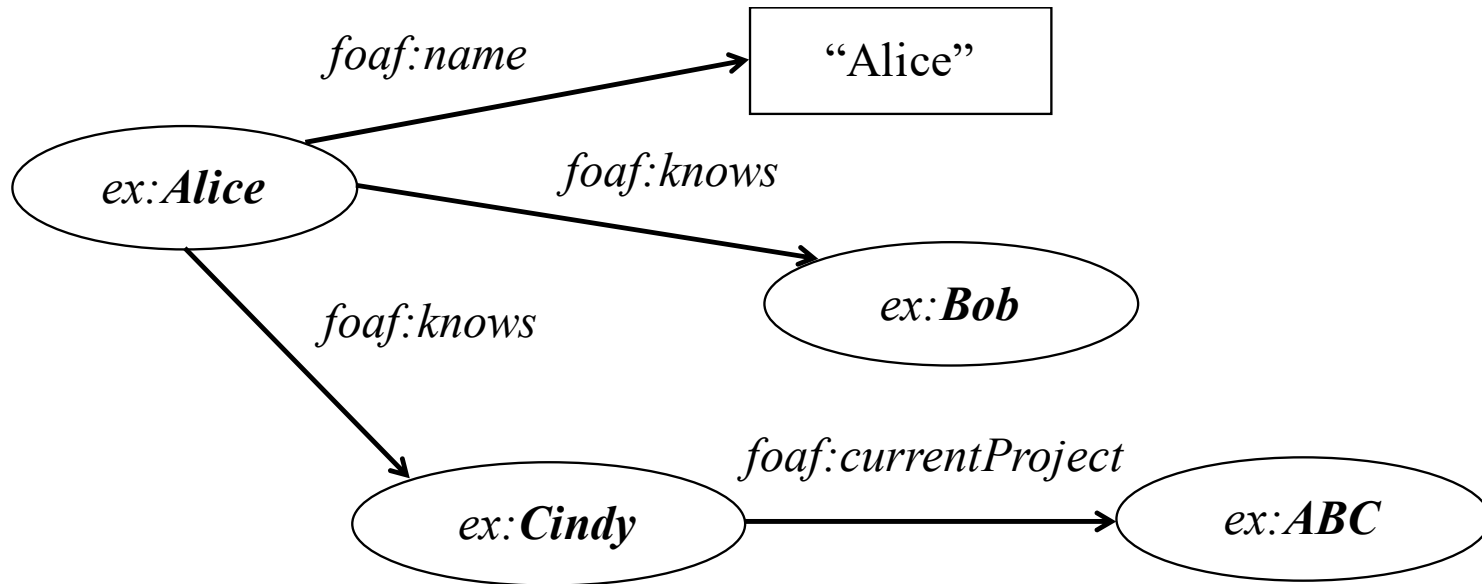
```
{ ex:Ryu foaf:knows _:a1 . _:a1 foaf:name "Ken" . }
```

can be abbreviated as

```
{ ex:Ryu foaf:knows [ foaf:name "Ken" ] . }
```

which can be read as *Ryu knows someone whose name is "Ken"*.

RDF Graph



```
ex:Alice    foaf:name      "Alice" .
ex:Alice    foaf:knows    ex:Bob .
ex:Alice    foaf:knows    ex:Cindy .
ex:Cindy    foaf:currentProject ex:ABC .
```

RDF Graph

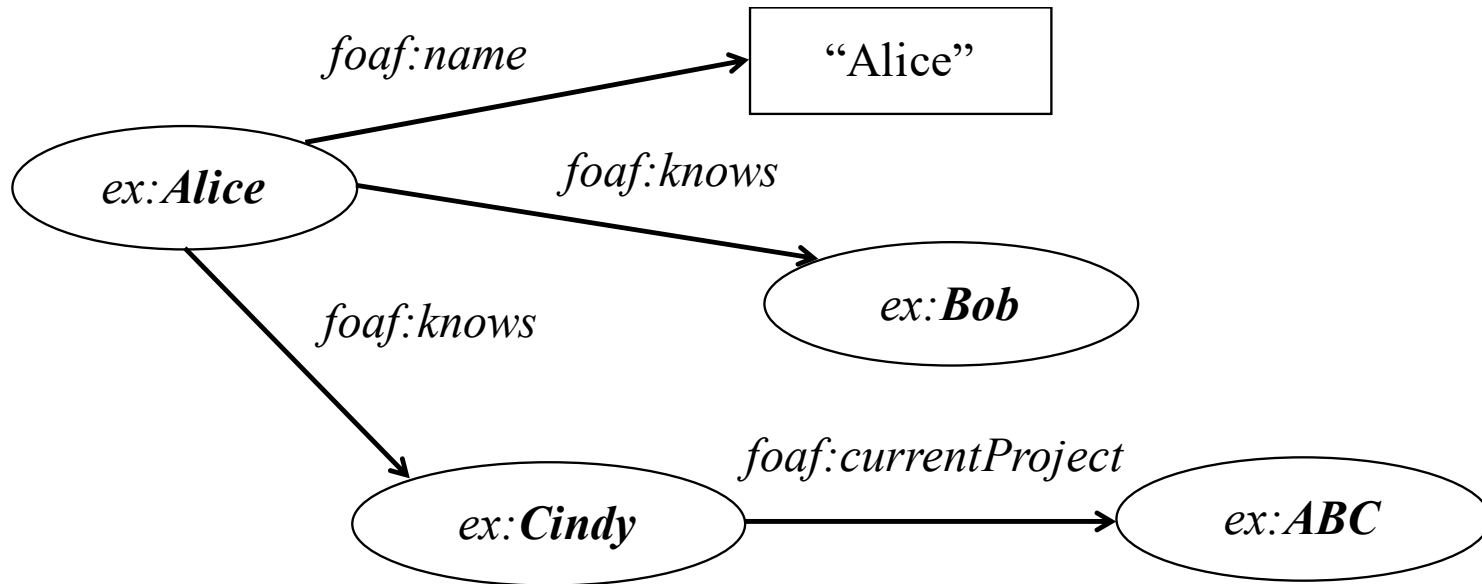
```
ex:Alice    foaf:name        "Alice" .
ex:Alice    foaf:knows       ex:Bob .
ex:Alice    foaf:knows       ex:Cindy .
ex:Cindy    foaf:currentProject ex:ABC .
```

can be abbreviated as the following expression

```
ex:Alice    foaf:name        "Alice" ;
            foaf:knows       ex:Bob , ex:Cindy .

ex:Cindy    foaf:currentProject ex:ABC .
```

RDF Graph



```
ex:Alice    foaf:name        "Alice" ;  
            foaf:knows       ex:Bob , ex: Cindy .  
  
ex:Cindy    foaf:currentProject  ex:ABC .
```

See Also ...

- **RDF**
 - <https://www.w3.org/TR/rdf-primer/>
- **Tools**
 - **Protégé**
 - <https://protege.stanford.edu/>
 - **OpenLink Virtuoso**
 - <https://virtuoso.openlinksw.com/>
 - **RDF4J**
 - <http://rdf4j.org/>



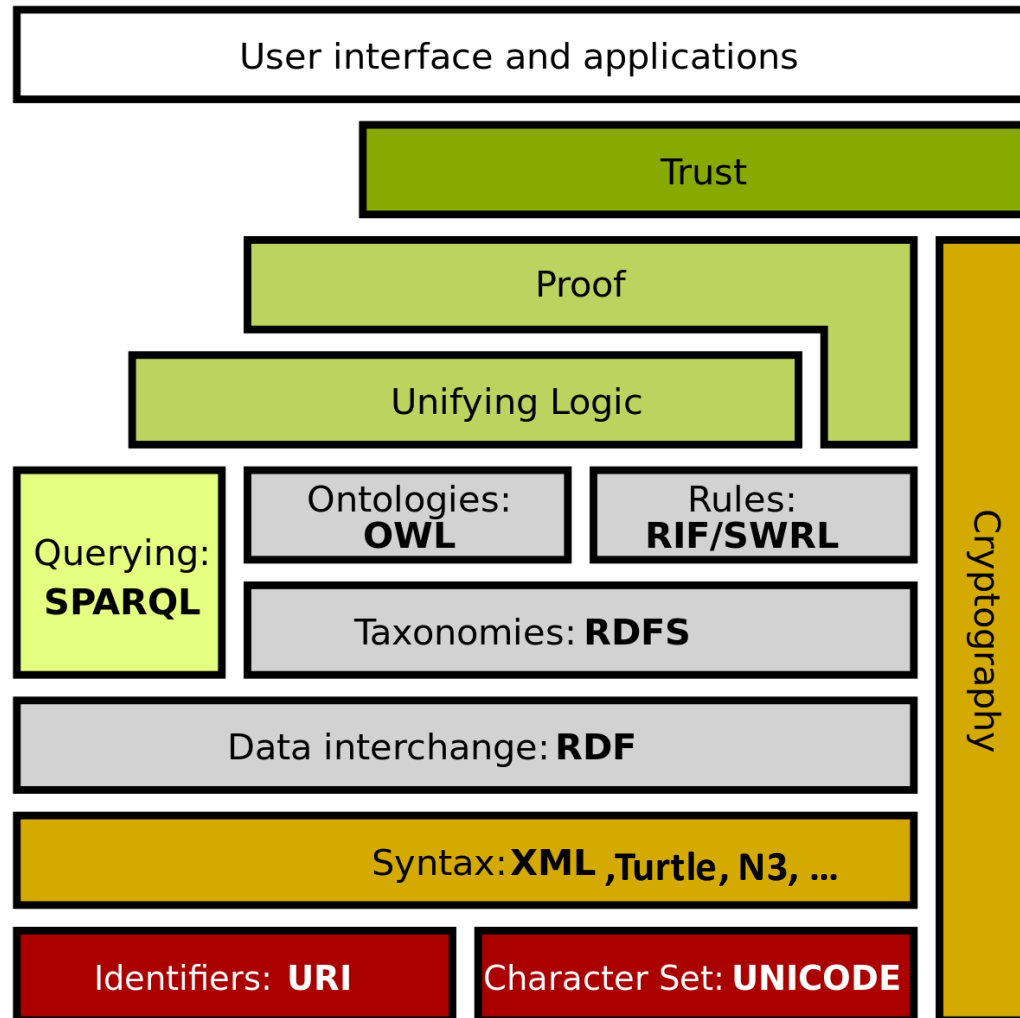


RDF Schema

The Interpretation on Ontology

- Without interpretation, an RDF statement is just a string of characters.
↓
7 bor
- Adding meaning to an RDF graph is a key feature of Semantic Web, and this feature makes an RDF graph become excellently machine-readable.
- Ontology is a formal, explicit specification of a shared conceptualization.

RDF Stack



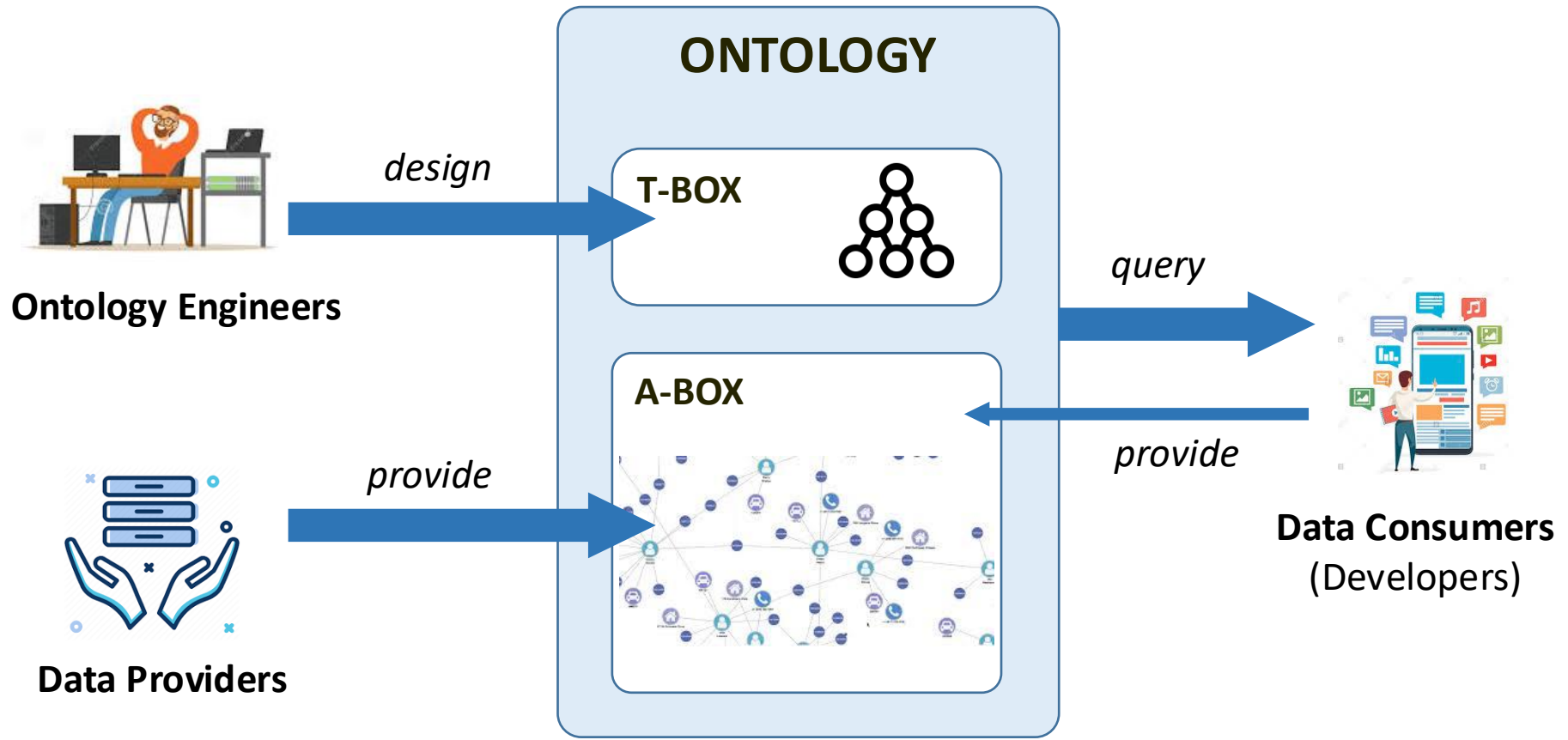
The Interpretation on Ontology

- Ontology is a formal, explicit specification of a shared conceptualization.
- Formal refers to **machine-readable**; explicit specification refers to concepts, properties, relations, constants, taxonomies, and axioms; and shared conceptualization refers to consensual knowledge of abstract model and real-world objects.
- **An RDF graph is also one part of ontology**. With the higher interpretation of RDF data with either RDF Schema or ontologies, the graph is more expressive.

RDF Schema (RDFS)

- RDFS provide basic entailments for interpreting ontologies. It mainly includes the interpretation of classes, properties, the hierarchies of classes, and the hierarchies of properties. **There are 13 RDFS deduction rules** for RDFS-Entailment, but we describe only rules that use in this thesis.
- It is noted that the statement $\langle x, \text{rdf:type}, C \rangle$ where x and C are any URIs and they can be interpreted that the **instance x** is a member of **a class or a set C** .
- In this section, the name of a class is recommended to begin with an **uppercase** letter, while the name of either an instance or a property begins with a **lowercase** letter.

Roles in LOD



Roles and Skills



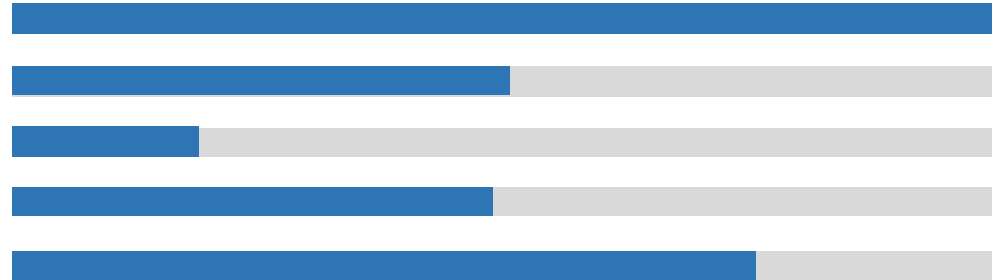
Ontology Engineers

RDF
RDFS & OWL
Reasoning
SPARQL
LOD



Data Providers

RDF
RDFS & OWL
Reasoning
SPARQL
LOD

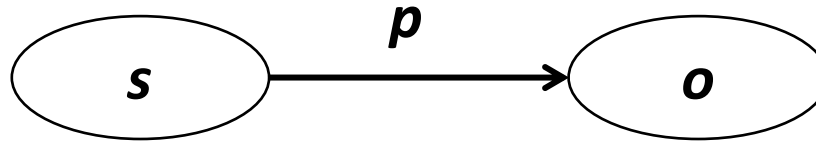


Data Consumers (developers)

RDF
RDFS & OWL
Reasoning
SPARQL
LOD



Domain



- **rdfs2:**

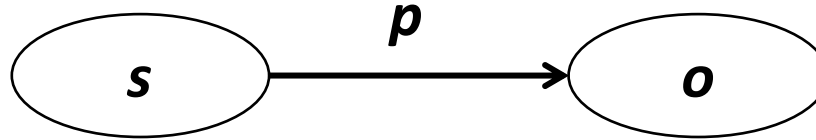
if $\langle p, \text{rdfs:domain}, C \rangle$ and $\langle s, p, o \rangle$,
then $\langle s, \text{rdf:type}, C \rangle$.

- **Example:**

if $\langle \text{:writes}, \text{rdfs:domain}, \text{foaf:Person} \rangle$
and $\langle \text{pp:John}, \text{:writes}, \text{ex:math101} \rangle$,

then $\langle \text{pp:John}, \text{rdf:type}, \text{foaf:Person} \rangle$.

Range



- **rdfs3:**

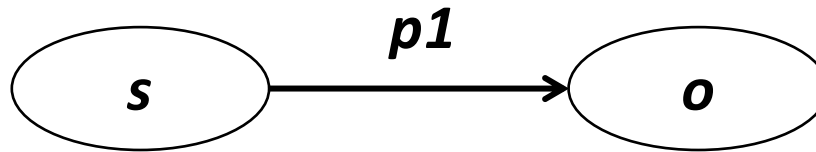
if $\langle p, \text{rdfs:range}, C \rangle$ and $\langle s, p, o \rangle$,
then $\langle o, \text{rdf:type}, C \rangle$.

- **Example**

if $\langle \text{:writes}, \text{rdfs:range}, \text{:TextBook} \rangle$
and $\langle \text{pp:John}, \text{:writes}, \text{ex:math101} \rangle$,

then $\langle \text{ex:math101}, \text{rdf:type}, \text{:TextBook} \rangle$.

Subproperties

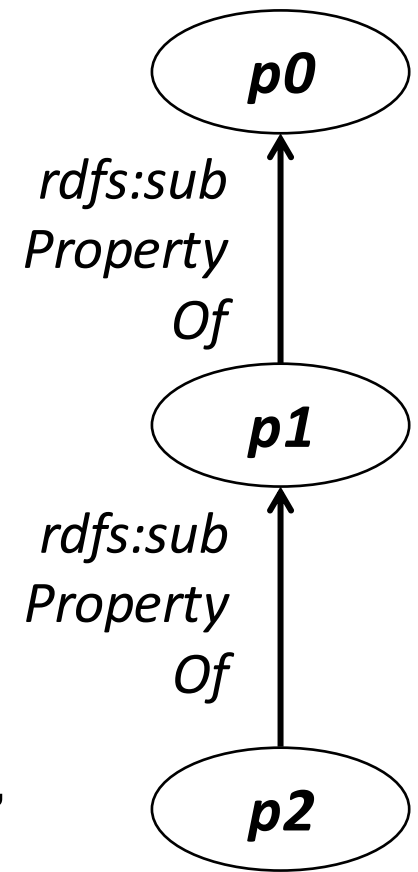


- **rdfs5:**

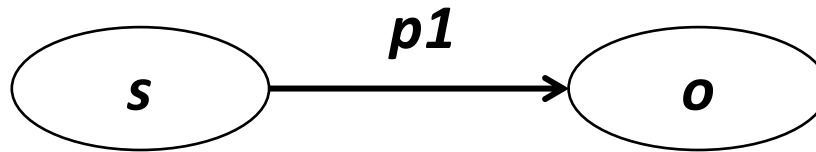
if $\langle p2, \text{rdfs:subPropertyOf}, p1 \rangle$
and $\langle p1, \text{rdfs:subPropertyOf}, p0 \rangle$,
then $\langle p2, \text{rdfs:subPropertyOf}, p0 \rangle$.

- **Example:**

if $\langle \text{:writes}, \text{rdfs:subPropertyOf}, \text{:create} \rangle$
and $\langle \text{:composes}, \text{rdfs:subPropertyOf}, \text{:write} \rangle$,
then $\langle \text{:composes}, \text{rdfs:subPropertyOf}, \text{:creates} \rangle$.



Subproperties



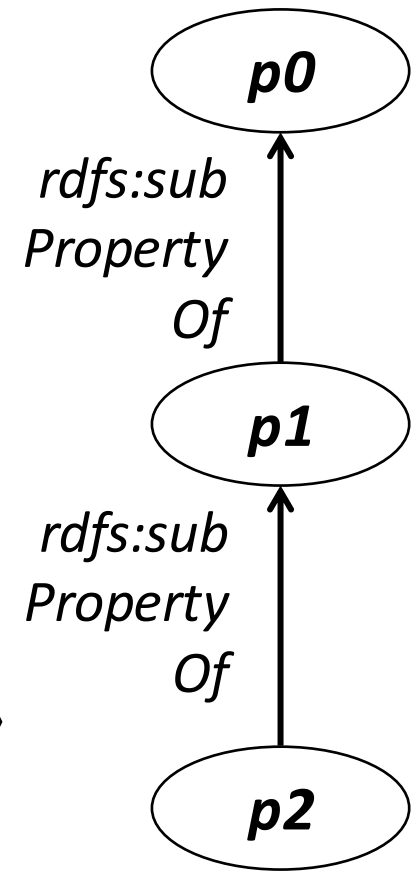
- **rdfs7:**

if $\langle p1, \text{rdfs:subPropertyOf}, p0 \rangle$
and $\langle s, p1, o \rangle$,
then $\langle s, p0, o \rangle$.

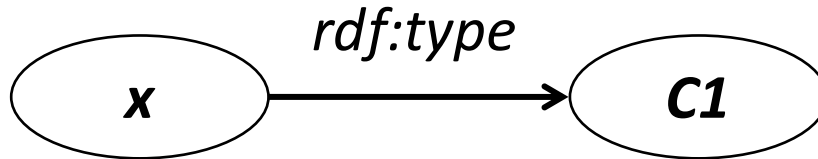
- **Example:**

if $\langle \text{:writes}, \text{rdfs:subPropertyOf}, \text{:creates} \rangle$
and $\langle \text{pp:John}, \text{:writes}, \text{ex:math101} \rangle$,

then $\langle \text{pp:John}, \text{:creates}, \text{ex:math101} \rangle$.



Subclasses

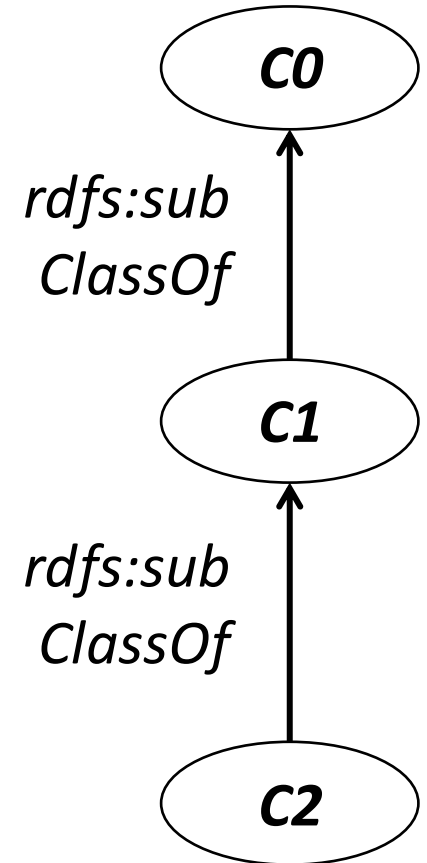


- **rdfs9:**

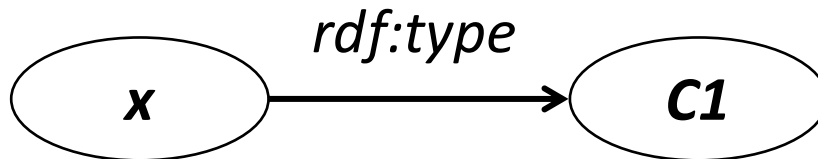
if $\langle C1, \text{rdfs:subClassOf}, C0 \rangle$
and $\langle x, \text{rdf:type}, C1 \rangle$,
then $\langle x, \text{rdf:type}, C0 \rangle$.

- **Example:**

if $\langle \text{:TextBook}, \text{rdfs:subClassOf}, \text{:Book} \rangle$
and $\langle \text{ex:math101}, \text{rdf:type}, \text{:TextBook} \rangle$,
then $\langle \text{ex:math101}, \text{rdf:type}, \text{:Book} \rangle$.



Subclasses

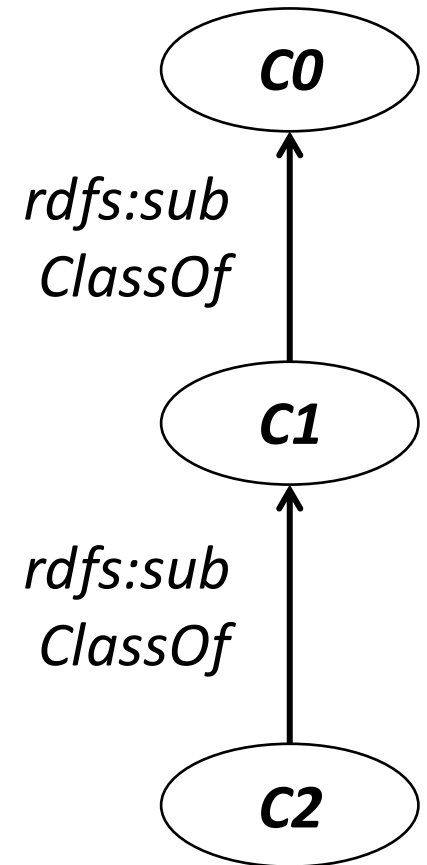


- **rdfs11:**

if $\langle C2, \text{rdfs:subClassOf}, C1 \rangle$
and $\langle C1, \text{rdfs:subClassOf}, C0 \rangle$,
then $\langle C2, \text{rdfs:subClassOf}, C0 \rangle$.

- **Example:**

if $\langle \text{:TextBook}, \text{rdfs:subClassOf}, \text{:Book} \rangle$
and $\langle \text{:Book}, \text{rdfs:subClassOf}, \text{:Document} \rangle$,
then $\langle \text{:TextBook}, \text{rdfs:subClassOf}, \text{:Document} \rangle$.



RDF Entailment

foaf:Person	rdfs:subClassOf	foaf:Agent .
foaf:homepage	rdfs:subPropertyOf	foaf:page ;
	rdfs:range	foaf:Document .

→ မှီခိုမှု ရှိသည်

TBox

ex:dan	rdf:type	foaf:Person ;
	foaf:homepage	<http://dan.info> .

ABox

RDF Entailment

foaf:Person	rdfs:subClassOf	foaf:Agent .	TBox
foaf:homepage	rdfs:subPropertyOf	foaf:page ;	
	rdfs:range	foaf:Document .	

ex:dan	rdf:type	foaf:Person ;	ABox
	foaf:homepage	<http://dan.info> .	



rdfs3:

if $\langle p, rdfs:range, C \rangle$ and $\langle s, p, o \rangle$,
then $\langle o, rdf:type, C \rangle$.



<http://dan.info> rdf:type foaf:Document .

RDF Entailment

foaf:Person	rdfs:subClassOf	foaf:Agent .	TBox
foaf:homepage	rdfs:subPropertyOf	foaf:page ;	
	rdfs:range	foaf:Document .	

ex:dan	rdf:type	foaf:Person ;	ABox
	foaf:homepage	<http://dan.info> .	



rdfs7:

if $\langle p1, rdfs:subPropertyOf, p0 \rangle$ and $\langle s, p1, o \rangle$,
then $\langle s, p0, o \rangle$.



ex:dan foaf:page <http://dan.info> .

RDF Entailment

<code>foaf:Person</code>	<code>rdfs:subClassOf</code>	<code>foaf:Agent</code> .	TBox
<code>foaf:homepage</code>	<code>rdfs:subPropertyOf</code>	<code>foaf:page</code> ;	
	<code>rdfs:range</code>	<code>foaf:Document</code> .	

<code>ex:dan</code>	<code>rdf:type</code>	<code>foaf:Person</code> ;	ABox
	<code>foaf:homepage</code>	<code><http://dan.info></code> .	



rdfs9:

if $\langle C1, rdfs:subClassOf, C0 \rangle$ and $\langle x, rdf:type, C1 \rangle$,
then $\langle x, rdf:type, C0 \rangle$.



`ex:dan` `rdf:type` `foaf:Agent` .

RDF Entailment

```
foaf:Person    rdfs:subClassOf    foaf:Agent .
foaf:homepage  rdfs:subPropertyOf  foaf:page ;
               rdfs:range          foaf:Document .
```

TBox

```
ex:dan         rdf:type            foaf:Person ;
               foaf:homepage       <http://dan.info> .
```

ABox



```
<http://dan.info>    rdf:type    foaf:Document .
```

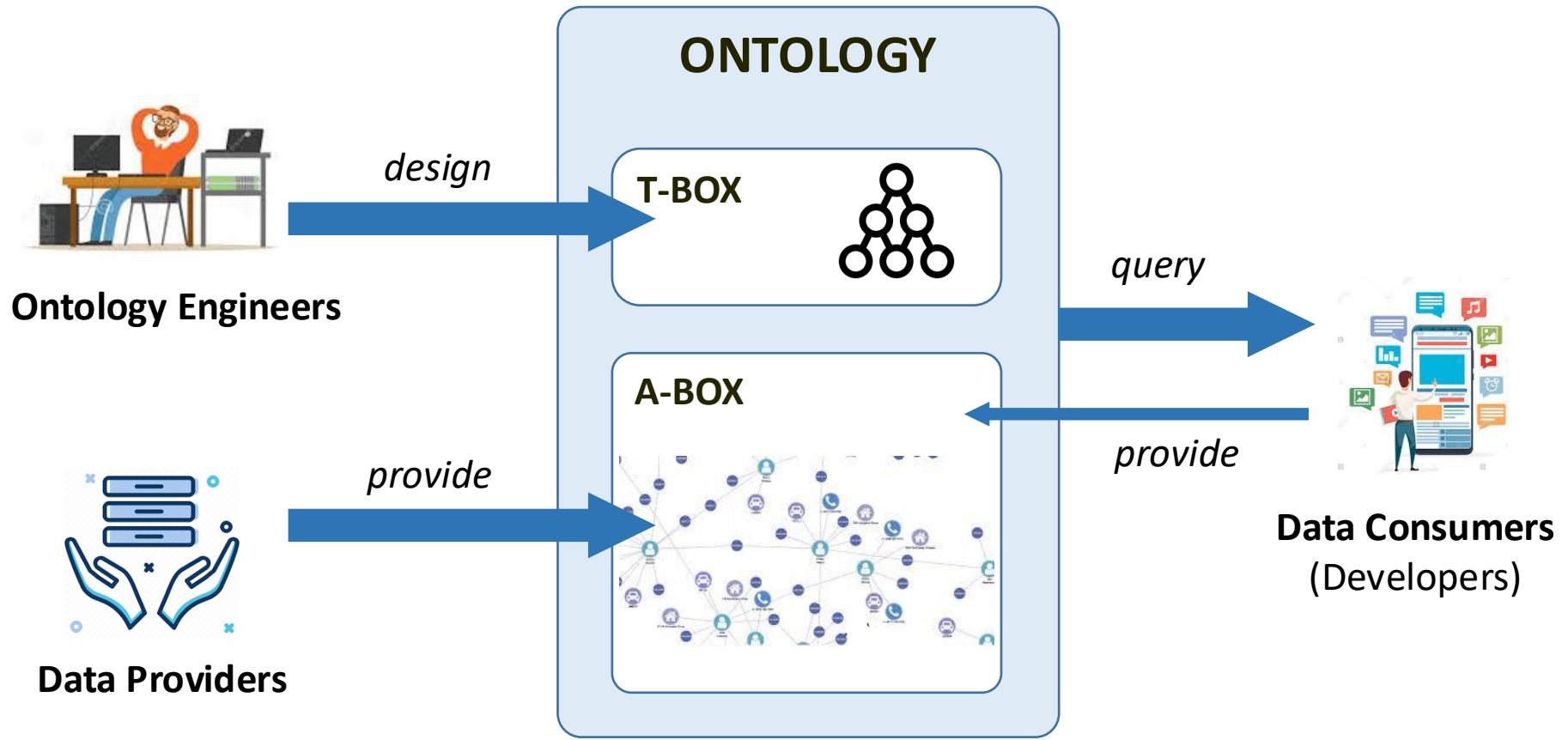
```
ex:dan    foaf:page    <http://dan.info> .
```

```
ex:dan    rdf:type     foaf:Agent .
```



Ontology

Roles in LOD



Roles and Skills



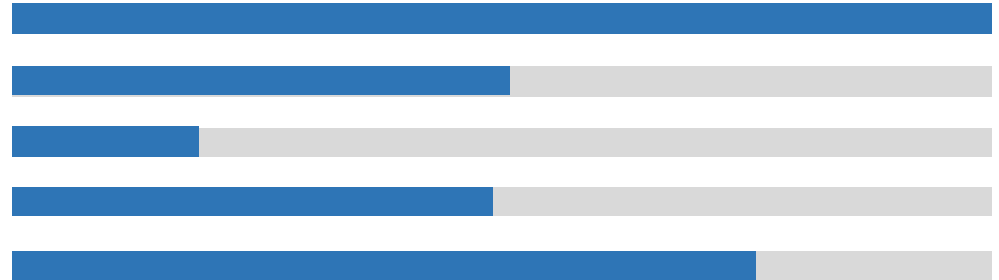
Ontology Engineers

RDF
RDFS & OWL
Reasoning
SPARQL
LOD



Data Providers

RDF
RDFS & OWL
Reasoning
SPARQL
LOD



Data Consumers (developers)

RDF
RDFS & OWL
Reasoning
SPARQL
LOD



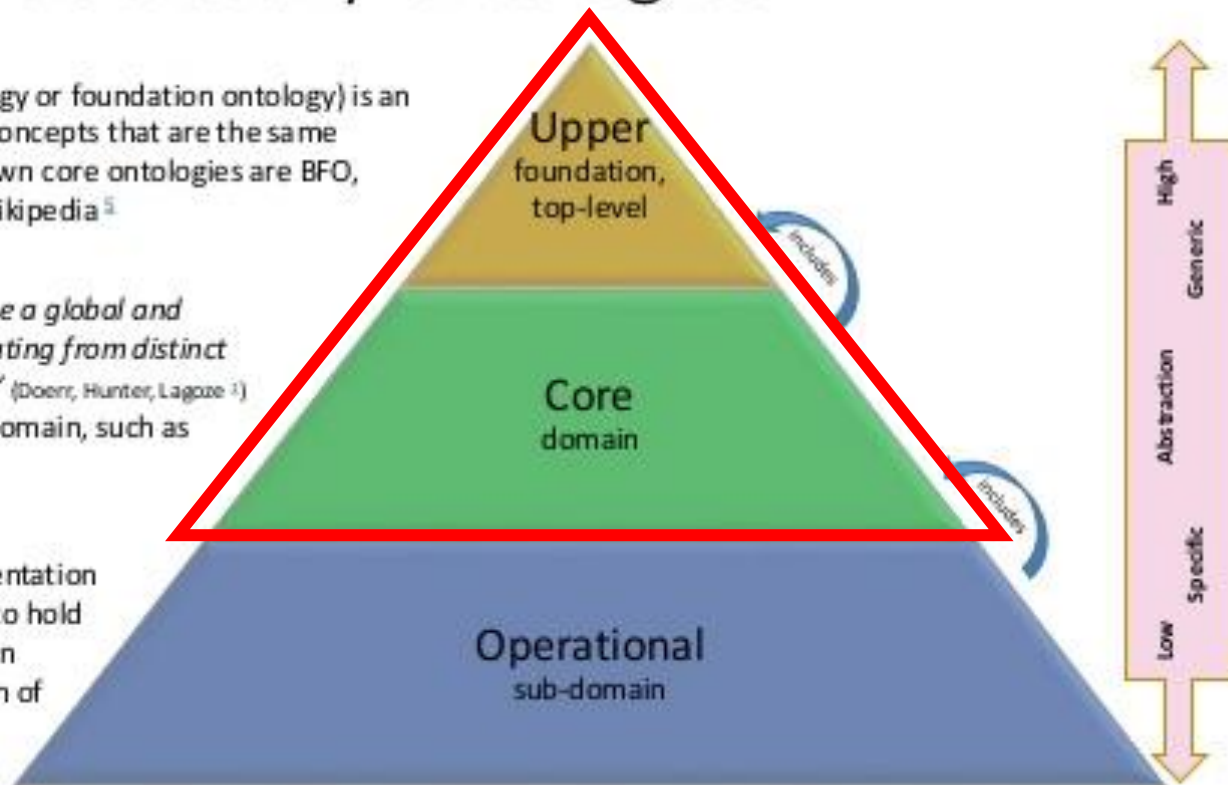
Ontology

Three main levels classify ontologies

An upper ontology (aka top-level ontology or foundation ontology) is an ontology which describes very general concepts that are the same across all knowledge domains. Well known core ontologies are BFO, GFO, DOLCE, SUMO and Dublin Core. Wikipedia ⁵

"The goal of a core ontology is to provide a global and extensible model into which data originating from distinct sources can be mapped and integrated." (Doerr, Hunter, Lagoze ¹)
The core ontology applies to a specific domain, such as biology, medical, Legal and Finance.

The Operational Ontology is an implementation of a core ontology. It is specific enough to hold source data. Hedge Fund Regulation is an operational ontology for the sub-domain of (alternative) investment management.



<http://livegate.com> © aspectdatamodels Inc. 2016

14

Web Ontology Language (OWL)

- OWL is a recommended standard for the modelling of ontologies. It gives higher interpretation over RDF and RDFS, so the representation of a knowledge graph becomes more expressive. There are many features and details in OWL, however, this review selects some features that are commonly used in this thesis, for example equality and property characteristics.
- It is firstly informed that every instance is belong to *owl:Thing*.

Equality

owl:sameAs

same-as:

if $\langle s1, \text{owl:sameAs}, s2 \rangle$ and

$\langle s1, p1, o1 \rangle$,

then $\langle s2, p1, o1 \rangle$.

if $\langle o1, \text{owl:sameAs}, o2 \rangle$ and

$\langle s1, p1, o1 \rangle$,

then $\langle s1, p1, o2 \rangle$.

Example:

if $\langle pp:9Arm, \text{owl:sameAs}, pp:Thananon \rangle$ and

$\langle pp:9Arm, \text{rdf:type}, :YouTuber \rangle$,

then $\langle pp:Thananon, \text{rdf:type}, :YouTuber \rangle$.

Equality

equivalence class:

if $\langle C1, \text{owl:equivalentClass}, C2 \rangle$ and $\langle x, \text{rdf:type}, C1 \rangle$,
then $\langle x, \text{rdf:type}, C2 \rangle$.

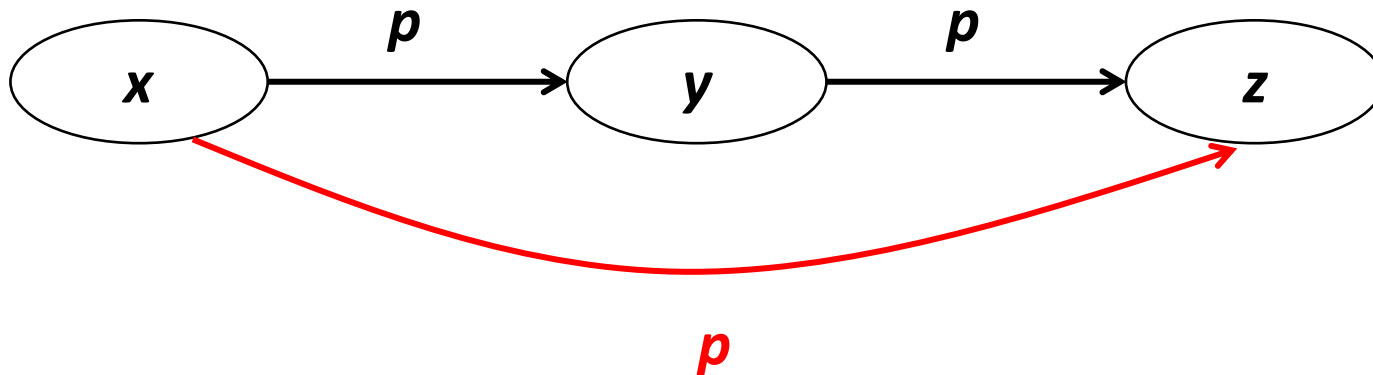
equivalence property:

if $\langle p1, \text{owl:equivalentProperty}, p2 \rangle$ and $\langle s1, p1, o1 \rangle$,
then $\langle s1, p2, o1 \rangle$.

Property Characteristics

transitive property:

if $\langle p, \text{rdf:type}, \text{owl:TransitiveProperty} \rangle$,
 $\langle x, p, y \rangle$, and $\langle y, p, z \rangle$,
then $\langle x, p, z \rangle$.

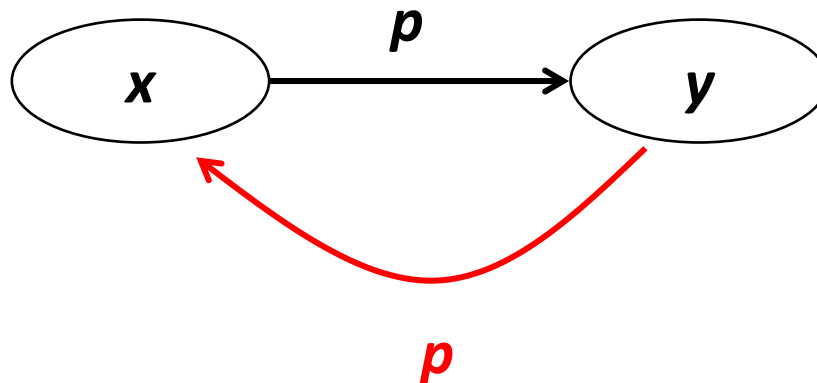


e.g. $p = :is_in$

Property Characteristics

symmetric property:

if $\langle p, \text{rdf:type}, \text{owl:SymmetricProperty} \rangle$ and $\langle x, p, y \rangle$
then $\langle y, p, x \rangle$.

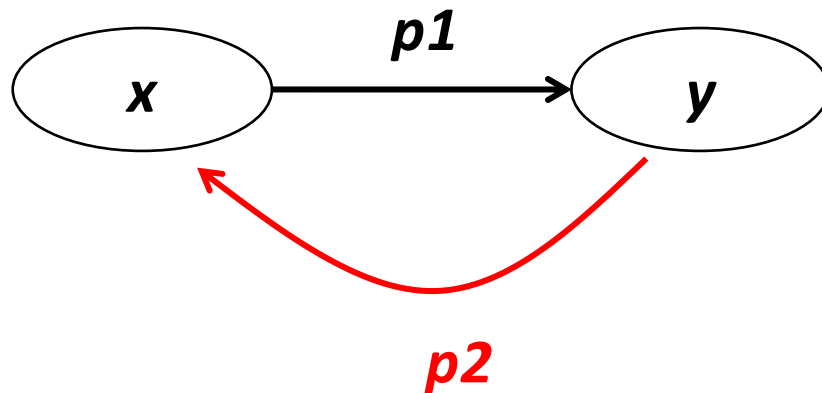


e.g. $p = :knows$

Property Characteristics

inverse property:

if $\langle p1, owl:inverseOf, p2 \rangle$, and $\langle x, p1, y \rangle$
then $\langle y, p2, x \rangle$.



e.g. $p1 = :has_child$
 $p2 = :has_parent$

OWL Reasoning

TBox

```
skos:broaderTransitive rdf:type owl:TransitiveProperty .  
skos:broader owl:reverseOf skos:narrower .
```

ABox

```
ex:football owl:sameAs ex:soccer .  
ex:football skos:broader ex:sport .
```

```
ex:dog skos:broaderTransitive ex:mammal .  
ex:mammal skos:broaderTransitive ex:animal .
```



```
ex:soccer skos:broader ex:sport .  
ex:sport skos:narrower ex:football .  
ex:sport skos:narrower ex:soccer .
```

```
ex:dog skos:broaderTransitive ex:animal .
```

DC-Term

- DC-Term (DCMI metadata term), which is the extension of Dublin Core Metadata, contains metadata for describing a documents and their relationships. For example
 - *dct:isVersionOf* is a property that identify the previous version of a term.
 - *dct:source* is a property that identify the reference.
- <http://dublincore.org/documents/dcmi-terms/>

FOAF → 15 நிமிடம்

- FOAF (Friend of a Friend) ontology provides vocabularies for describing humans, organizations, and documents, and relationships among them. For example
 - *foaf:Person* is a class of persons, and it is the sub class of *foaf:Agent*.
 - *foaf:knows* is a symmetric property that identifies persons knowing each other.
 - *foaf:depiction* is a property that points to a picture that is the instance of the class *foaf:Image*.
- <http://xmlns.com/foaf/spec/>

SKOS

↳ Dictionary

- SKOS (Simple Knowledge Organization System) ontology provides a model for expressing the basic structure and content of concept schemes. For example
 - *skos:broader* is a property that identifies the board concept of a given concept.
 - *skos:broaderTransitive* is a transitive property that identifies the transitively board concept of a given concept.
 - *skos:narrower* is a property that identifies the narrow concept of a given concept and it is the inverse property of *skos:broader*.
 - *skos:narrowerTransitive* is a property that identifies the transitively narrow concept of a given concept and it is the inverse property of *skos:broaderTransitive*.
- <https://www.w3.org/TR/skos-primer/>

See Also ...

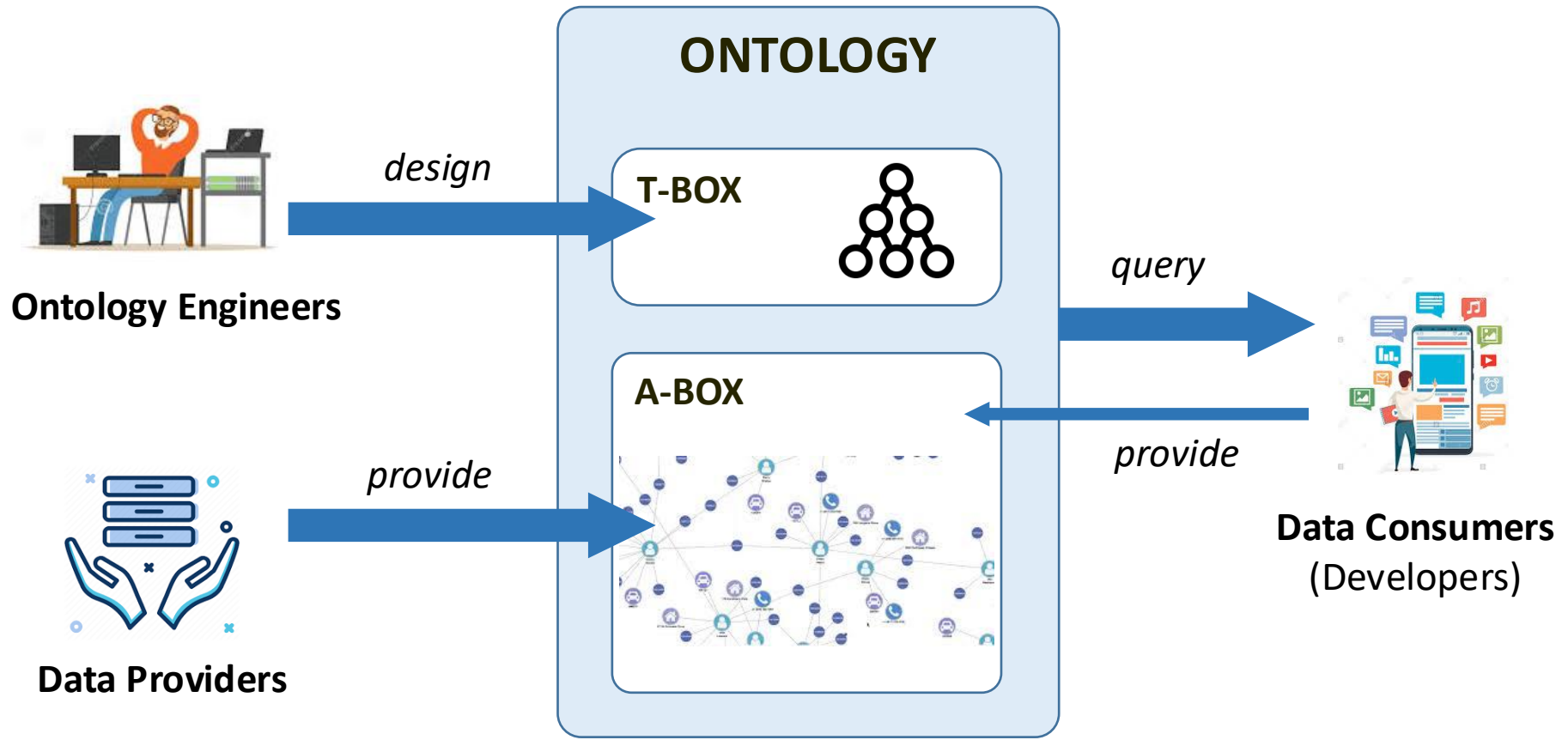
- RDFS
 - <https://www.w3.org/TR/rdf-schema/>
- OWL
 - <https://www.w3.org/TR/2012/REC-owl2-primer-20121211/>
- List of Ontologies
 - <http://semanticweb.org/wiki/Ontology>





Designing Ontology

Roles in LOD



Roles and Skills



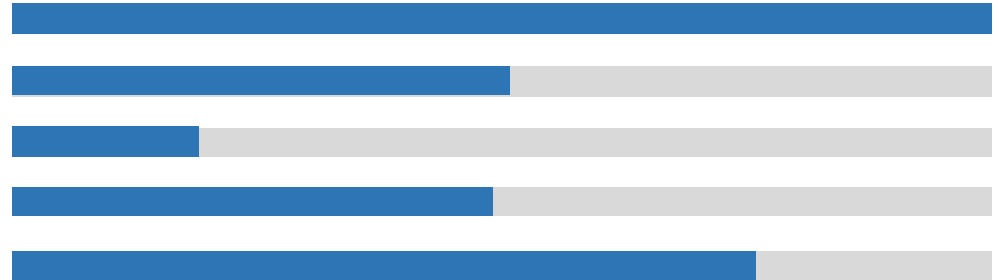
Ontology Engineers

RDF
RDFS & OWL
Reasoning
SPARQL
LOD



Data Providers

RDF
RDFS & OWL
Reasoning
SPARQL
LOD



Data Consumers (developers)

RDF
RDFS & OWL
Reasoning
SPARQL
LOD



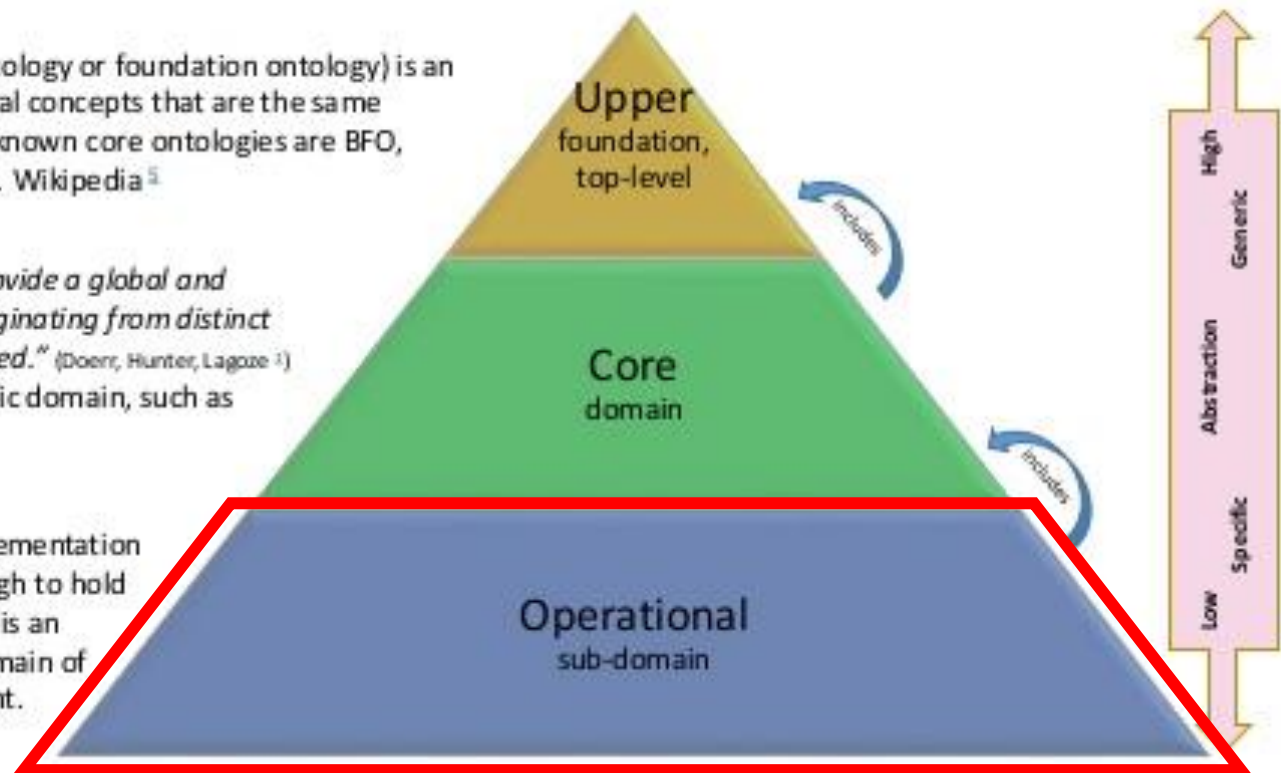
Ontology

Three main levels classify ontologies

An upper ontology (aka top-level ontology or foundation ontology) is an ontology which describes very general concepts that are the same across all knowledge domains. Well known core ontologies are BFO, GFO, DOLCE, SUMO and Dublin Core. Wikipedia ⁵

"The goal of a core ontology is to provide a global and extensible model into which data originating from distinct sources can be mapped and integrated." (Doerr, Hunter, Lagoze ¹)
The core ontology applies to a specific domain, such as biology, medical, Legal and Finance.

The Operational Ontology is an implementation of a core ontology. It is specific enough to hold source data. Hedge Fund Regulation is an operational ontology for the sub-domain of (alternative) investment management.

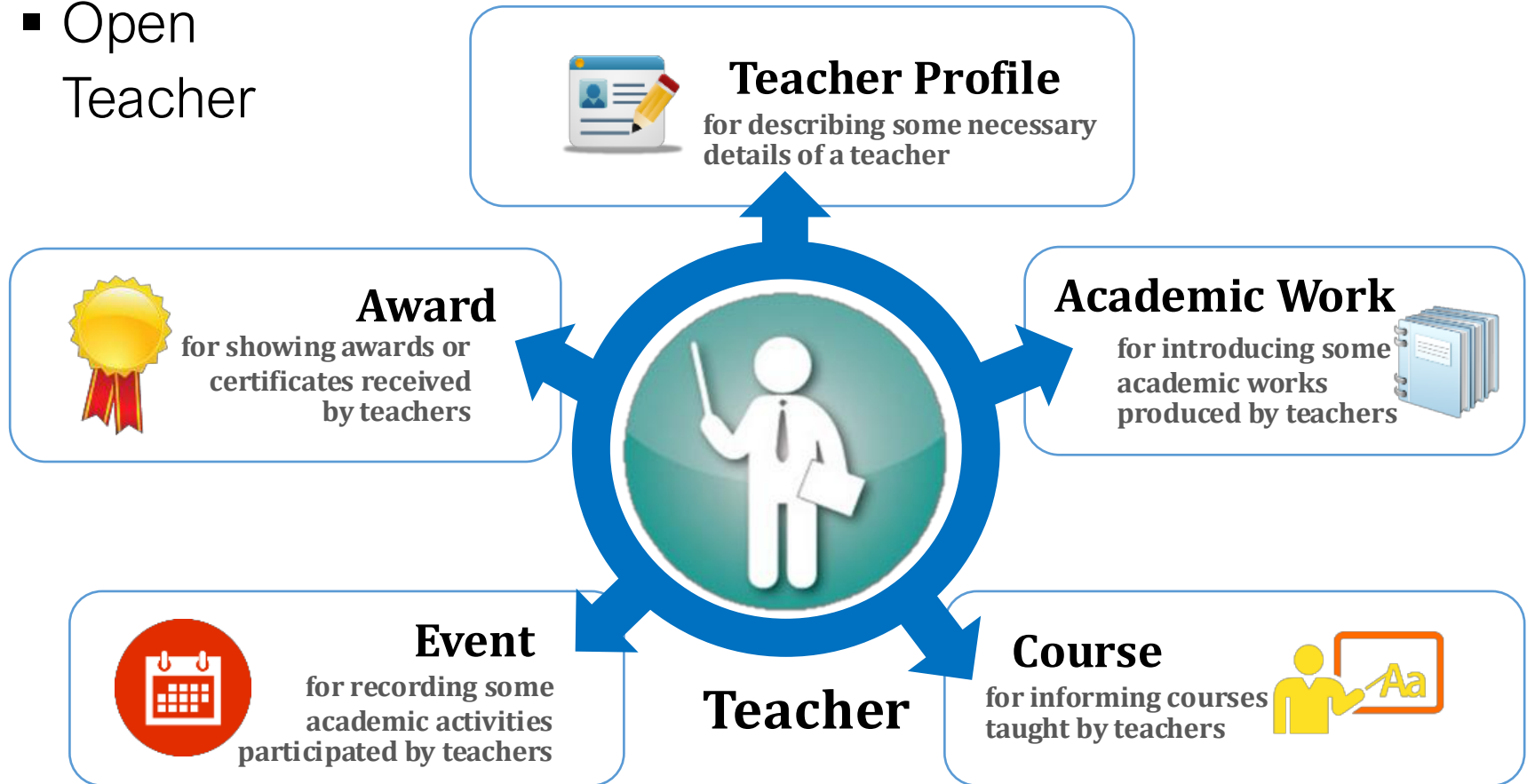


<http://livegate.com> © aspectdatamodels inc 2016

14

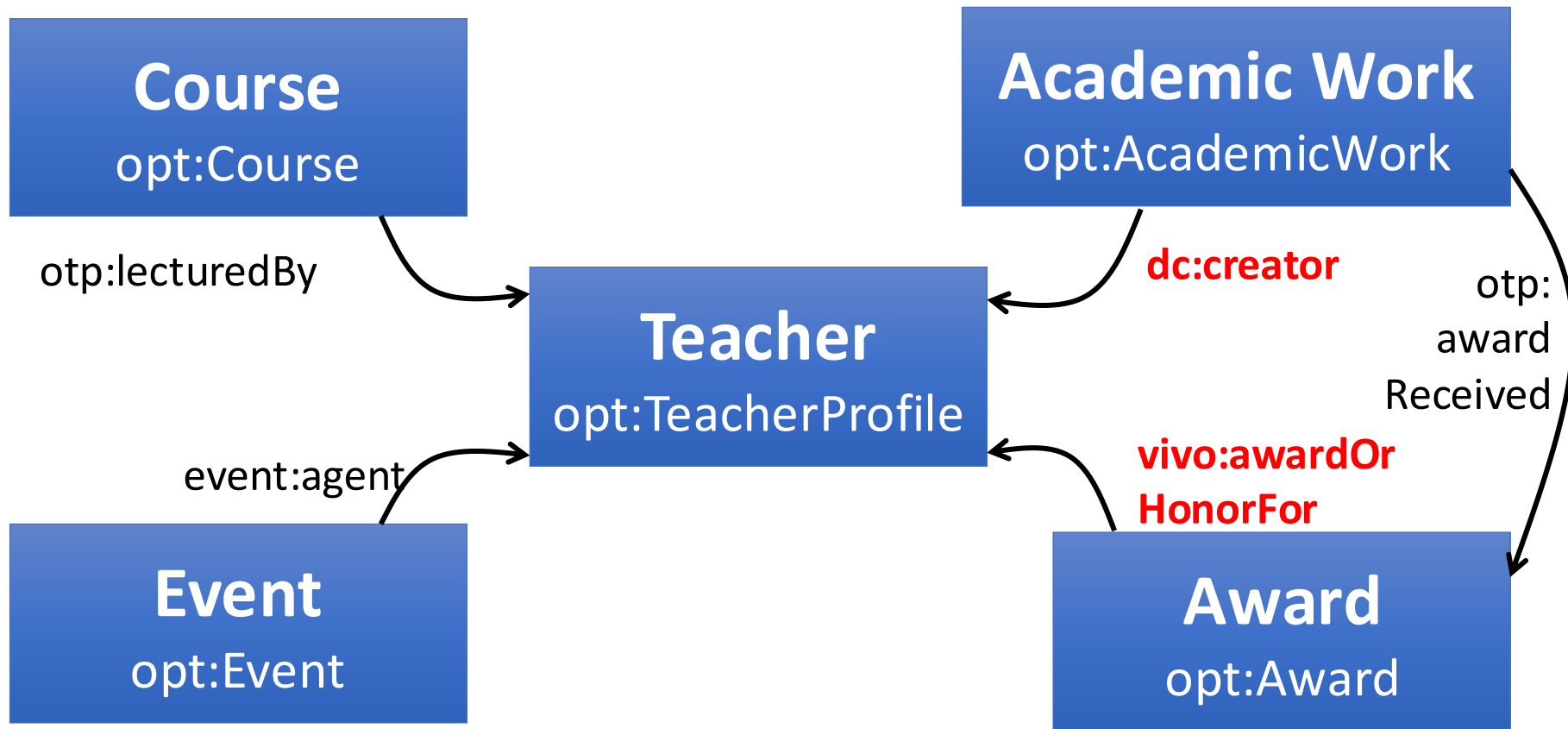
Application Profile

- Open Teacher

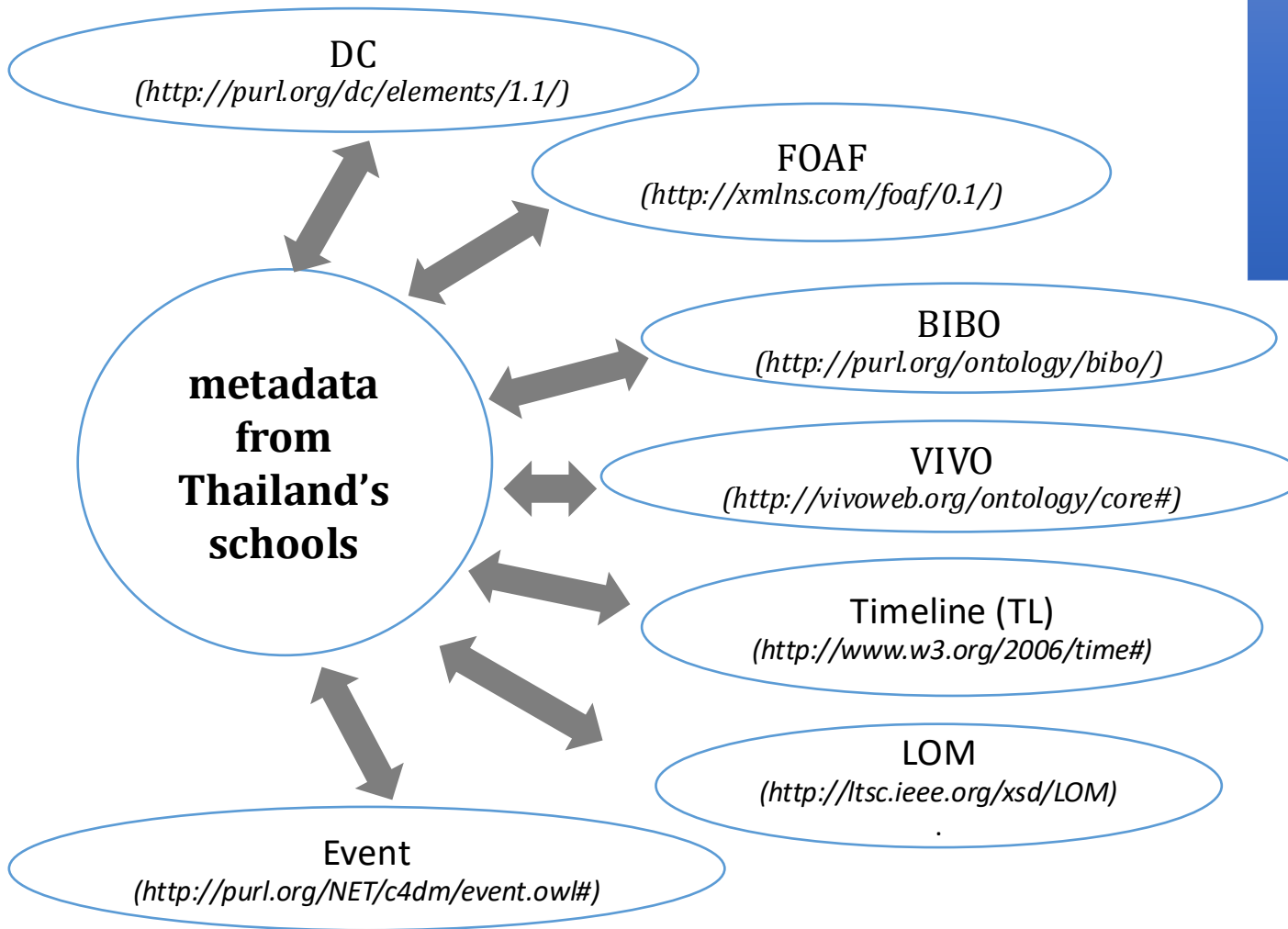


Application Profile

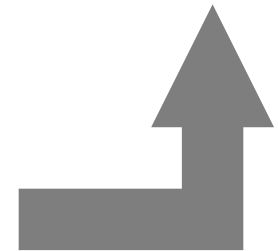
- Domain Model



Metadata



**Common
Application
Profile**



Teacher Profile



- opt:TeacherProfile

Property	URI	Example Value
Identifier	vivo:identifier	"1976072189"
Preferred Title	vivo:preferredTitle	"Ajarn"
First name	foaf:firstName	"Prayut"
Last name	foaf:lastName	"Chawuthai"
Academic Standing	vivo:hasTeacherRole	otp:ExpertTeacher
Academic Degree	otp:hasAcademicDegree	otp:Bachelor
Email	foaf:mbox	"ajprayut@gmail.com"
Website	foaf:homepage	https://profile.com/prayut
Field of Expertise	otp:expertIn	otp:Art, otp:Math
Area	lom:discipline	otp:Art

Course



- opt:Course

Property	URI	Example Value
Course Code	<i>dc:identifier</i>	"ART-101"
Course Name	<i>dc:title</i>	"Fundamentals of Art 1"
Lecturer	<i>otp:courseLecturedBy</i>	ex:Prayut
Begin Date	<i>tl:beginAtDateTime</i>	"2011-06-01T09:00:00Z" ^{^^xsd:dateTime}
End date	<i>tl:endAtDateTime</i>	"2011-09-30T10:00:00Z" ^{^^xsd:dateTime}
Learning Area	<i>lom:discipline</i>	otp:Art
Grade	<i>lom:educationLevel</i>	otp:Grade1

Example

```
ex:Prayut    rdf:type          otp:TeacherProfile ;
              vivo:identifier    "1976072189" ;
              foaf:firstName     "Prayut" ;
              vivo:hasTeacherRole otp:ExpertTeacher ;
              lom:discipline      otp:Art, otp:Math ;
              otp:expertIn        otp:WaterColor .

ex:art101    rdf:type          otp:Course
              dc:title          "Fundamentals of Art 1" ;
              dc:identifier      "ART-101" ;
              otp:courseLecturedBy ex:Prayut ;
              lom:discipline      otp:Art ;
              lom:educationLevel  otp:Grade1 .
```

See Also ...

- **List of Ontologies**
 - <http://semanticweb.org/wiki/Ontology>
- **Open Teacher**
 - https://link.springer.com/chapter/10.1007/978-3-319-12823-8_15
- **Linked Taxonomic Knowledge**
 - <https://content.iospress.com/articles/semantic-web/sw192>
- **Protégé**
 - <https://protege.stanford.edu/>





Reasoning

Semantic Web Reasoning

- RDFS and OWL provide necessary rules to entail an RDF graph.
- In case some specific requirements beyond RDFS and OWL are needed, developers can create some Semantic Web rules.
- For creating own rules, the book of Semantic Web programming recommended to **use Apache Jena** as a reasoning engine that is a library of Java.

Semantic Web Rules

- For example, in case the condition “An uncle is a brother of one’s father” is defined, it can be simply expressed that:

If $\langle x, :hasFather, y \rangle$ and $\langle y, :hasBrother, z \rangle$,
then $\langle x, :hasUncle, z \rangle$.

This rule can be written in a dialog program as follows:

```
hasFather(?x, ?y) ∧ hasBrother(?y, ?z) → hasUncle(?x, ?z)
```

JENA

- Semantic Web Rule

```
hasFather(?x, ?y) ∧ hasBrother(?y, ?z) → hasUncle(?x, ?z)
```



- JENA Rule

```
[rule_identify_uncle:  
  (?x :hasFather ?y),  
  (?y :hasBrother ?z)  
-> (?x :hasUncle ?z) ]
```

HEAD

- RDF Data

```
ex:john    :hasFather   ex:smith .  
ex:smith   :hasBrother  ex:adam .
```



- JENA Rule

```
[rule_identify_uncle:  
  (?x    :hasFather   ?y) ,  
  (?y    :hasBrother  ?z)  
-> (?x    :hasUncle   ?z) ]
```



- Result

```
ex:john    :hasUncle   ex:adam .
```

See Also ...

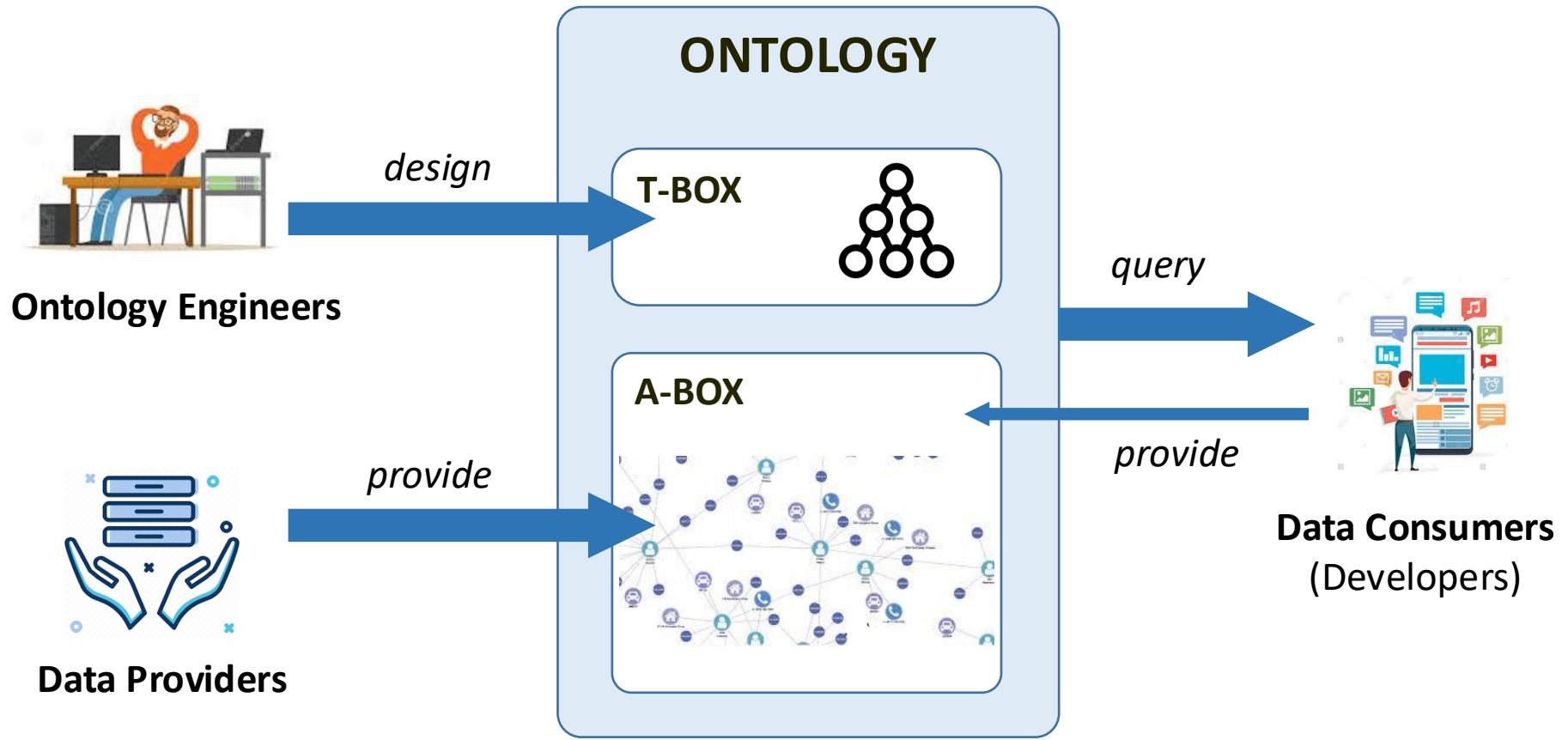
- **Semantic Web Rules**
 - <https://www.w3.org/Submission/SWRL/>
- **JENA**
 - <https://jena.apache.org/documentation/inference/>
- **Protégé**
 - <https://protege.stanford.edu/>





Query

Roles in LOD



Roles and Skills



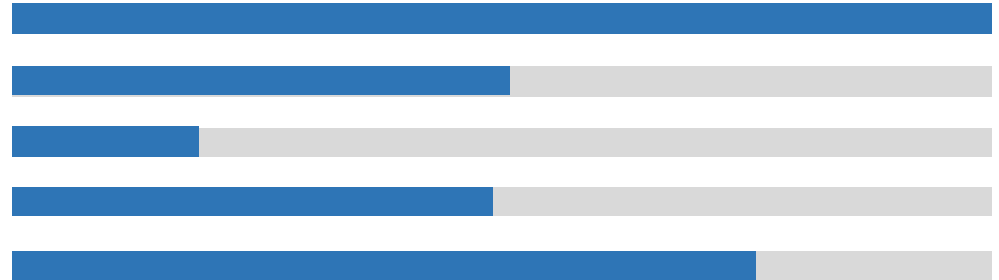
Ontology Engineers

RDF
RDFS & OWL
Reasoning
SPARQL
LOD



Data Providers

RDF
RDFS & OWL
Reasoning
SPARQL
LOD



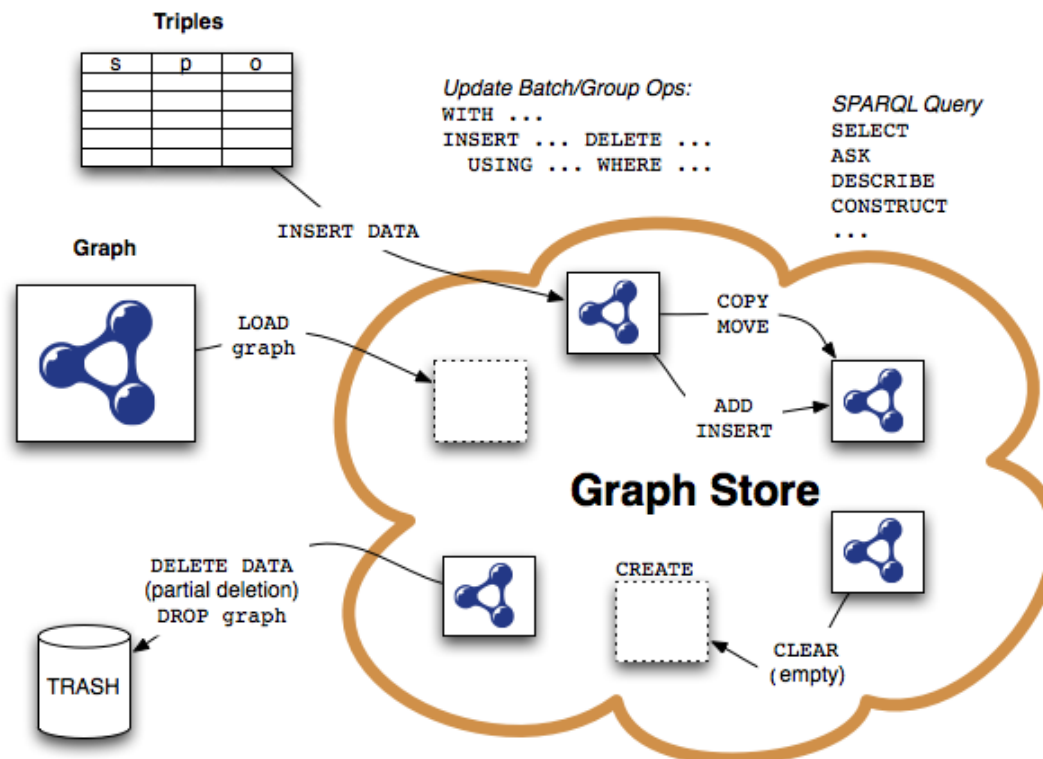
Data Consumers (developers)

RDF
RDFS & OWL
Reasoning
SPARQL
LOD



SPARQL

- SPARQL (SPARQL Protocol and RDF Query Language) is a protocol and query language for querying RDF data. The syntax includes prefixes, scheme or result, and condition.



SPARQL

1.2.1 Namespaces

In this document, examples assume the following namespace prefix bindings unless otherwise stated:

Prefix	IRI
rdf:	http://www.w3.org/1999/02/22-rdf-syntax-ns#
rdfs:	http://www.w3.org/2000/01/rdf-schema#
xsd:	http://www.w3.org/2001/XMLSchema#
fn:	http://www.w3.org/2005/xpath-functions#

SPARQL : Simple

1.2.2 Data Descriptions

This document uses the [Turtle](#) [TURTLE] data format to show each triple explicitly. Turtle allows IRIs to be abbreviated with prefixes:

```
@prefix dc:    <http://purl.org/dc/elements/1.1/> .  
@prefix :     <http://example.org/book/> .  
:book1  dc:title  "SPARQL Tutorial" .
```

1.2.3 Result Descriptions

Result sets are illustrated in tabular form.

x	y	z
"Alice"	<http://example/a>	

SPARQL : Simple

Data:

```
<http://example.org/book/book1> <http://purl.org/dc/elements/1.1/title> "SPARQL Tutorial" .
```

Query:

```
SELECT ?title
WHERE
{
  <http://example.org/book/book1> <http://purl.org/dc/elements/1.1/title> ?title .
}
```

This query, on the data above, has one solution:

Query Result:

title
"SPARQL Tutorial"

SPARQL : 2++ Conditions

Data:

```
@prefix foaf: <http://xmlns.com/foaf/0.1/> .  
  
_:a foaf:name "Johnny Lee Outlaw" .  
_:a foaf:mbox <mailto:jlow@example.com> .  
_:b foaf:name "Peter Goodguy" .  
_:b foaf:mbox <mailto:peter@example.org> .  
_:c foaf:mbox <mailto:carol@example.org> .
```

Query:

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>  
SELECT ?name ?mbox  
WHERE  
{ ?x foaf:name ?name .  
  ?x foaf:mbox ?mbox }
```

Query Result:

name	mbox
"Johnny Lee Outlaw"	<mailto:jlow@example.com>
"Peter Goodguy"	<mailto:peter@example.org>

SPARQL: Literal

```
@prefix dt:    <http://example.org/datatype#> .
@prefix ns:    <http://example.org/ns#> .
@prefix :      <http://example.org/ns#> .
@prefix xsd:   <http://www.w3.org/2001/XMLSchema#> .

:x  ns:p      "cat"@en .
:y  ns:p      "42"^^xsd:integer .
:z  ns:p      "abc"^^dt:specialDatatype .
```

```
SELECT ?v WHERE { ?v ?p "cat" }
```

v

```
SELECT ?v WHERE { ?v ?p "cat"@en }
```

v
<http://example.org/ns#x>

```
SELECT ?v WHERE { ?v ?p 42 }
```

v
<http://example.org/ns#y>

SPARQL: Filter

```
@prefix dc:    <http://purl.org/dc/elements/1.1/> .
@prefix :      <http://example.org/book/> .
@prefix ns:    <http://example.org/ns#> .

:book1  dc:title  "SPARQL Tutorial" .
:book1  ns:price  42 .
:book2  dc:title  "The Semantic Web" .
:book2  ns:price  23 .
```

```
PREFIX  dc:  <http://purl.org/dc/elements/1.1/>
SELECT  ?title
WHERE   { ?x dc:title ?title
          FILTER regex(?title, "^SPARQL")
        }
```

title
"SPARQL Tutorial"

SPARQL: Filter

Query:

```
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX ns: <http://example.org/ns#>
SELECT ?title ?price
WHERE { ?x ns:price ?price .
        FILTER (?price < 30.5)
        ?x dc:title ?title . }
```

Query Result:

title	price
"The Semantic Web"	23

SPARQL: Optional

```
@prefix foaf:      <http://xmlns.com/foaf/0.1/> .
@prefix rdf:       <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

_:a  rdf:type      foaf:Person .
_:a  foaf:name     "Alice" .
_:a  foaf:mbox     <mailto:alice@example.com> .
_:a  foaf:mbox     <mailto:alice@work.example> .

_:b  rdf:type      foaf:Person .
_:b  foaf:name     "Bob" .
```

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT ?name ?mbox
WHERE { ?x foaf:name ?name .
       OPTIONAL { ?x foaf:mbox ?mbox }
}
```

name	mbox
"Alice"	<mailto:alice@example.com>
"Alice"	<mailto:alice@work.example>
"Bob"	

SPARQL: Optional

```
@prefix foaf:      <http://xmlns.com/foaf/0.1/> .

_:a  foaf:name      "Alice" .
_:a  foaf:homepage  <http://work.example.org/alice/> .

_:b  foaf:name      "Bob" .
_:b  foaf:mbox       <mailto:bob@work.example> .
```

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT ?name ?mbox ?hpage
WHERE {
  ?x foaf:name ?name .
      OPTIONAL { ?x foaf:mbox ?mbox } .
      OPTIONAL { ?x foaf:homepage ?hpage }
}
```

name	mbox	hpage
"Alice"		<http://work.example.org/alice/>
"Bob"	<mailto:bob@work.example>	

SPARQL: Duplicated Results

```
@prefix foaf: <http://xmlns.com/foaf/0.1/> .  
  
_:x foaf:name "Alice" .  
_:x foaf:mbox <mailto:alice@example.com> .  
  
_:y foaf:name "Alice" .  
_:y foaf:mbox <mailto:asmith@example.com> .  
  
_:z foaf:name "Alice" .  
_:z foaf:mbox <mailto:alice.smith@example.com> .
```

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>  
SELECT ?name WHERE { ?x foaf:name ?name }
```

name
"Alice"
"Alice"
"Alice"

SPARQL: Limit, Offset

```
PREFIX foaf:    <http://xmlns.com/foaf/0.1/>

SELECT ?name
WHERE { ?x foaf:name ?name }
LIMIT 20
```

```
PREFIX foaf:    <http://xmlns.com/foaf/0.1/>

SELECT  ?name
WHERE   { ?x foaf:name ?name }
ORDER BY ?name
LIMIT   5
OFFSET  10
```

SPARQL: Select

- RDF Data

```
ex:Movie      rdfs:subClassOf    ex:Entertainment .
ex:TVSeries   rdfs:subClassOf    ex:Entertainment .

ex:starwarsVII    rdf:type        ex:Movie ;
                  ex:starring     ex:daisy, ex:adam .

ex:gameOfThrones rdf:type        ex:TVSeries .
```



- SPARQL Statement

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX ex:  <http://www.example.org/ex-terms/>

SELECT ?ent WHERE { ?ent rdf:type ex:Entertainment .}
```



- Query Result

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+		
	?ent	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+		
	http://www.example.org/ex-terms/starwarsVII	
	http://www.example.org/ex-terms/gameOfThrones	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+		

SPARQL: Construct

- RDF Data

```
ex:Movie      rdfs:subClassOf    ex:Entertainment .
ex:TVSeries   rdfs:subClassOf    ex:Entertainment .

ex:starwarsVII    rdf:type        ex:Movie ;
                  ex:starring     ex:daisy, ex:adam .

ex:gameOfThrones rdf:type        ex:TVSeries .
```



- SPARQL Statement

```
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX ex:   <http://www.example.org/ex-terms/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

CONSTRUCT { ?a      foaf:knows      ?b .}
WHERE      { ?ent    ex:starring     ?a , ?b .}
```

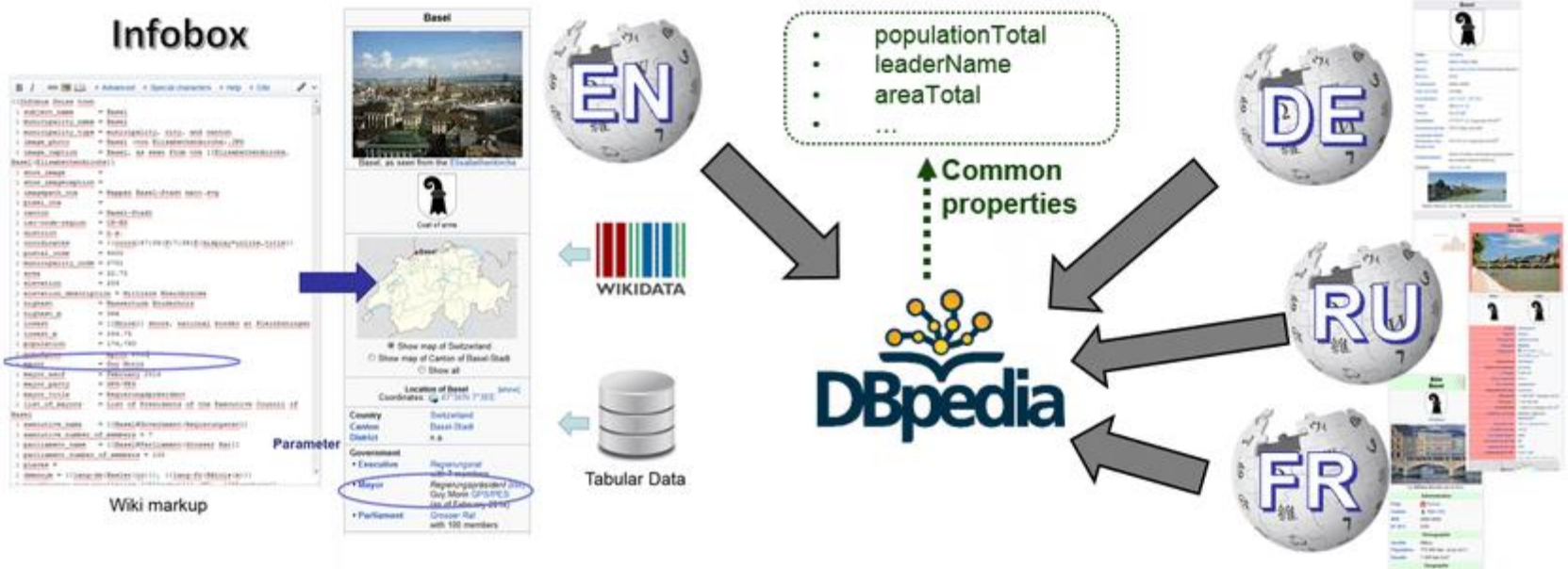


- Query Result

```
<http://www.example.org/ex-terms/daisy>
  <http://xmlns.com/foaf/0.1/knows>
    <http://www.example.org/ex-terms/adam> .
```

DBpedia

- WikiPedia Inforbox → Ontology



https://www.researchgate.net/publication/321690179_Relative_Quality_and_Popularity_Evaluation_of_Multilingual_Wikipedia_Articles

DBpedia SPARQL

- <https://dbpedia.org/sparql>

Virtuoso SPARQL Query Editor

[About](#) | [Namespace Prefixes](#) | [Inference rules](#) | [RDF views](#) | [iSPARQ](#)

Default Data Set Name (Graph IRI)

Query Text

```
PREFIX ont: <http://dbpedia.org/ontology/>
SELECT * WHERE {?comp rdf:type ont:Company. }
```

(Security restrictions of this server do not allow you to retrieve remote RDF data, see [details](#).)

Results Format:

Execution timeout:

milliseconds *(values less than 1000 are ignored)*

Options:

- ☒ Strict checking of void variables
- ☐ Log debug info at the end of output (has no effect on some queries and output formats)
- ☐ Generate SPARQL compilation report (instead of executing the query)

(The result can only be sent back to browser, not saved on the server, see [details](#))

DBpedia SPARQL

- <https://dbpedia.org/sparql>

comp
http://dbpedia.org/resource/Hongsalmun
http://dbpedia.org/resource/3Com
http://dbpedia.org/resource/7-Eleven
http://dbpedia.org/resource/7_Up
http://dbpedia.org/resource/AIOD_leads
http://dbpedia.org/resource/Aardman_Animations
http://dbpedia.org/resource/About.com
http://dbpedia.org/resource/Acorn_Computers
http://dbpedia.org/resource/Activision
http://dbpedia.org/resource/Ad_Lib,_Inc.
http://dbpedia.org/resource/Adnams_Brewery
http://dbpedia.org/resource/Advance-fee_scam
http://dbpedia.org/resource/Advanced_Data_Communication_Control_Procedures

Let's try

- Begin your query with

```
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX : <http://dbpedia.org/resource/>
PREFIX dbpedia2: <http://dbpedia.org/property/>
PREFIX dbpedia: <http://dbpedia.org/>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
```

- They can be copied from
 - <http://dbpedia.org/snorql/>

Let's try

```
PREFIX ont: <http://dbpedia.org/ontology/>
```

```
SELECT ?a WHERE {<http://dbpedia.org/resource/Apple_Inc.> ont:product ?a }
```

```
SELECT ?a ?b WHERE {<http://dbpedia.org/resource/Apple_Inc.> ?a ?b }
```

```
PREFIX ont: <http://dbpedia.org/ontology/>
```

```
SELECT * WHERE { ?a rdf:type ont:Company. ?a ont:product :IPad }
```

```
PREFIX ont: <http://dbpedia.org/ontology/>
```

```
SELECT * WHERE { ?a ont:product ?b. ?b dbpedia2:weight ?c }
```

```
PREFIX ont: <http://dbpedia.org/ontology/>
```

```
SELECT ?a ?b ?c
```

```
WHERE { ?a ont:product ?b. ?b dbpedia2:weight ?c. FILTER regex( ?c , "Wi-Fi model:") }
```

```
PREFIX ont: <http://dbpedia.org/ontology/>
```

```
SELECT ?a ?b ?c WHERE { ?a ont:product ?b. OPTIONAL { ?b dbpedia2:weight ?c. } }
```

Let's try

```
SELECT ?a ?b WHERE {?a foaf:name ?b}
```

```
SELECT ?a ?b ?c WHERE {?a foaf:name ?b. ?a rdfs:label ?c.}
```

```
SELECT ?a ?b ?c WHERE {?a foaf:name ?b; rdfs:label ?c.}
```

```
SELECT ?a ?b ?c WHERE {?a foaf:name ?b. ?b rdfs:label ?c.}
```

```
SELECT ?a ?b ?c WHERE {?a dbpedia2:populationDensityKm ?b.}
```

```
SELECT ?a ?b ?c WHERE {?a dbpedia2:populationDensityKm ?b. FILTER ( ?b > 30).}
```

```
SELECT ?a ?b ?c WHERE {?a dbpedia2:populationDensityKm ?b. FILTER ( ?b > 30). ?a  
rdfs:comment ?c}
```

```
SELECT ?a ?b WHERE {?a foaf:name ?b}
```

Let's try

```
PREFIX ont: <http://dbpedia.org/ontology/>
SELECT ?a ?b WHERE {<http://dbpedia.org/resource/Apple_Inc.> ont:product ?a
OPTIONAL {?a dbpedia2:memory ?b}}
```

```
PREFIX ont: <http://dbpedia.org/ontology/>
SELECT ?a ?c WHERE {<http://dbpedia.org/resource/Apple_Inc.> ont:product ?a
OPTIONAL {?a dbpedia2:license ?c}}
```

```
PREFIX ont: <http://dbpedia.org/ontology/>
SELECT ?a ?b ?c WHERE {<http://dbpedia.org/resource/Apple_Inc.> ont:product ?a
.OPTIONAL {?a dbpedia2:memory ?b}. OPTIONAL {?a dbpedia2:license ?c}}
```

```
PREFIX ont: <http://dbpedia.org/ontology/>
SELECT ?a ?c WHERE {<http://dbpedia.org/resource/Apple_Inc.> ont:product ?a
OPTIONAL {?a dbpedia2:license ?c}}
```

```
PREFIX ont: <http://dbpedia.org/ontology/>
SELECT ?a ?b ?c WHERE {<http://dbpedia.org/resource/Apple_Inc.> ont:product ?a
.OPTIONAL {?a dbpedia2:memory ?b}. OPTIONAL {?a dbpedia2:license ?c}}
```

Let's try

```
PREFIX ont: <http://dbpedia.org/ontology/>
```

```
SELECT ?a ?b
```

```
WHERE { <http://dbpedia.org/resource/Apple_Inc.> ont:product ?a. {?a  
dbpedia2:memory ?b } }
```

```
PREFIX ont: <http://dbpedia.org/ontology/>
```

```
select * where { {[:Microsoft ont:foundationPerson ?a }
```

```
UNION [:Microsoft ont:industry ?a]]. OPTIONAL {?a ont:birthPlace ?c} }
```

```
SELECT ?src ?age
```

```
WHERE { GRAPH ?src {?x foaf:name "Bill Gates"@en. ?x dbpedia2:years ?age } }
```

```
CONSTRUCT {?x ?y ?z}
```

```
WHERE { GRAPH <http://dbpedia.org> {?x ?y ?z . ?x foaf:name "Bill Gates"@en } }
```

```
DESCRIBE ?x WHERE { GRAPH <http://dbpedia.org> {?x foaf:name "Bill Gates"@en } }
```

```
ASK {?x dbpedia2:name "iPad"@en }
```

See Also ...

- **SPARQL**
 - <https://www.w3.org/TR/rdf-sparql-query/>
- **DBpedia**
 - <https://wiki.dbpedia.org/>
- **DBpedia SPARQL**
 - <https://dbpedia.org/sparql>
- **Dbpedia SNORQL**
 - <http://dbpedia.org/snorql/>





LOD

Linked Open Data

Linked Open Data

- Linked Data is about using the Web to connect related data that wasn't previously linked, or using the Web to lower the barriers to linking data currently linked using other methods.
- More specifically, Wikipedia defines Linked Data as "a term used to describe a recommended best practice for exposing, sharing, and connecting pieces of data, information, and knowledge on the **Semantic Web** using URIs and RDF."
- <http://linkeddata.org/>

5-star Data

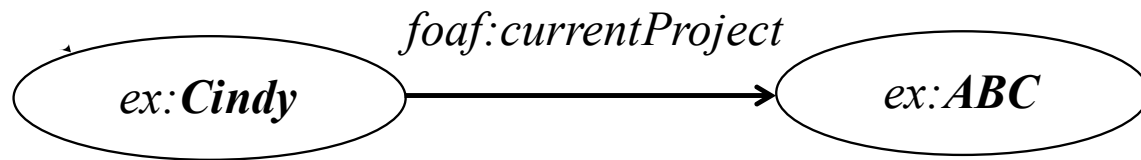
LINKED DATA

- ★ On the web, open license
- ★ ★ Machine-readable data
- ★ ★ ★ Non-proprietary format
- ★ ★ ★ ★ RDF standards
- ★ ★ ★ ★ ★ Linked RDF

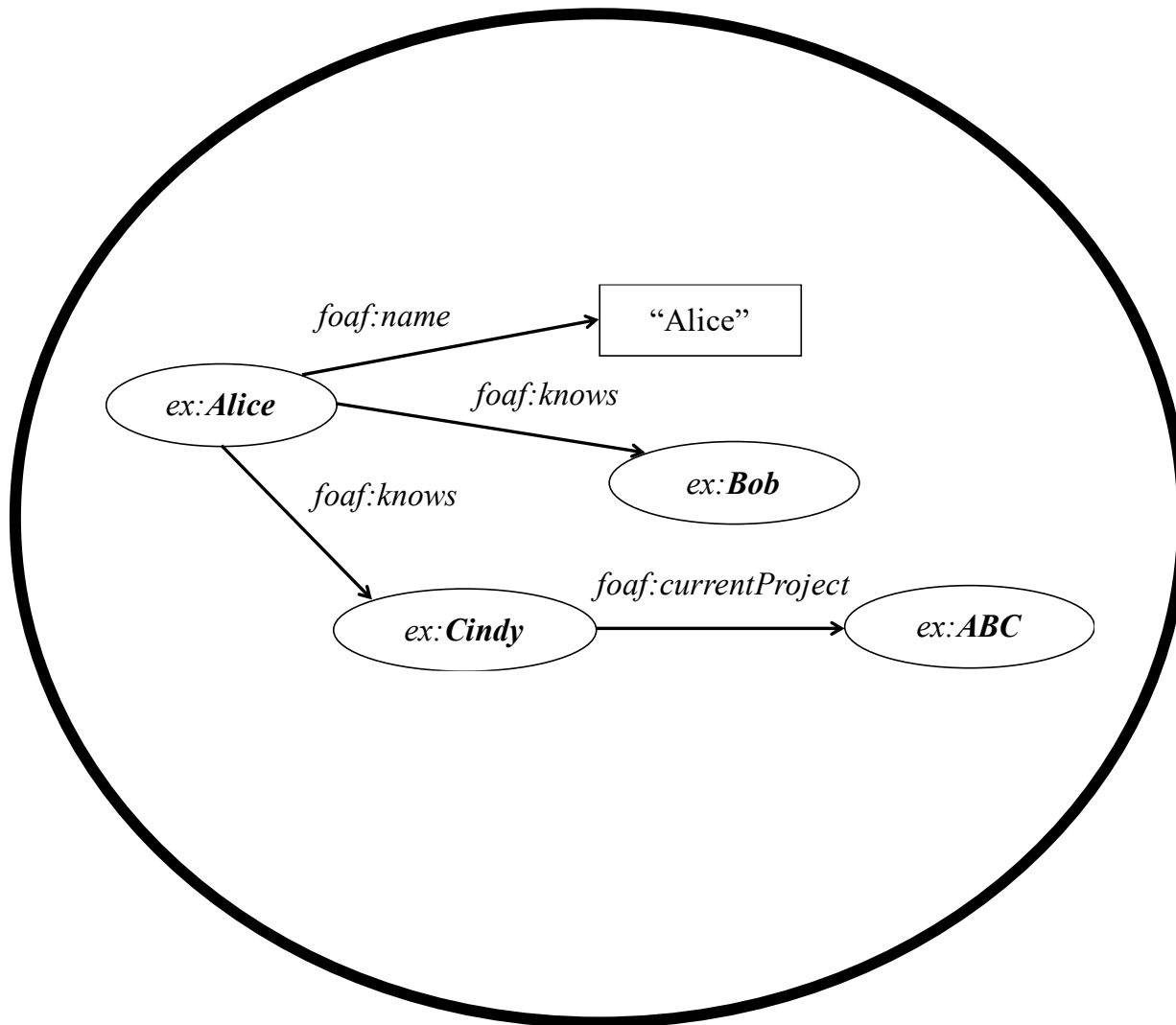
IS YOUR DATA 5 ★ ?

Open License	e.g. Online PDF
Reusable	e.g. Excel
Open Format	e.g. CSV
URI's	RDF files
Linked Data	Ontologies

Triple

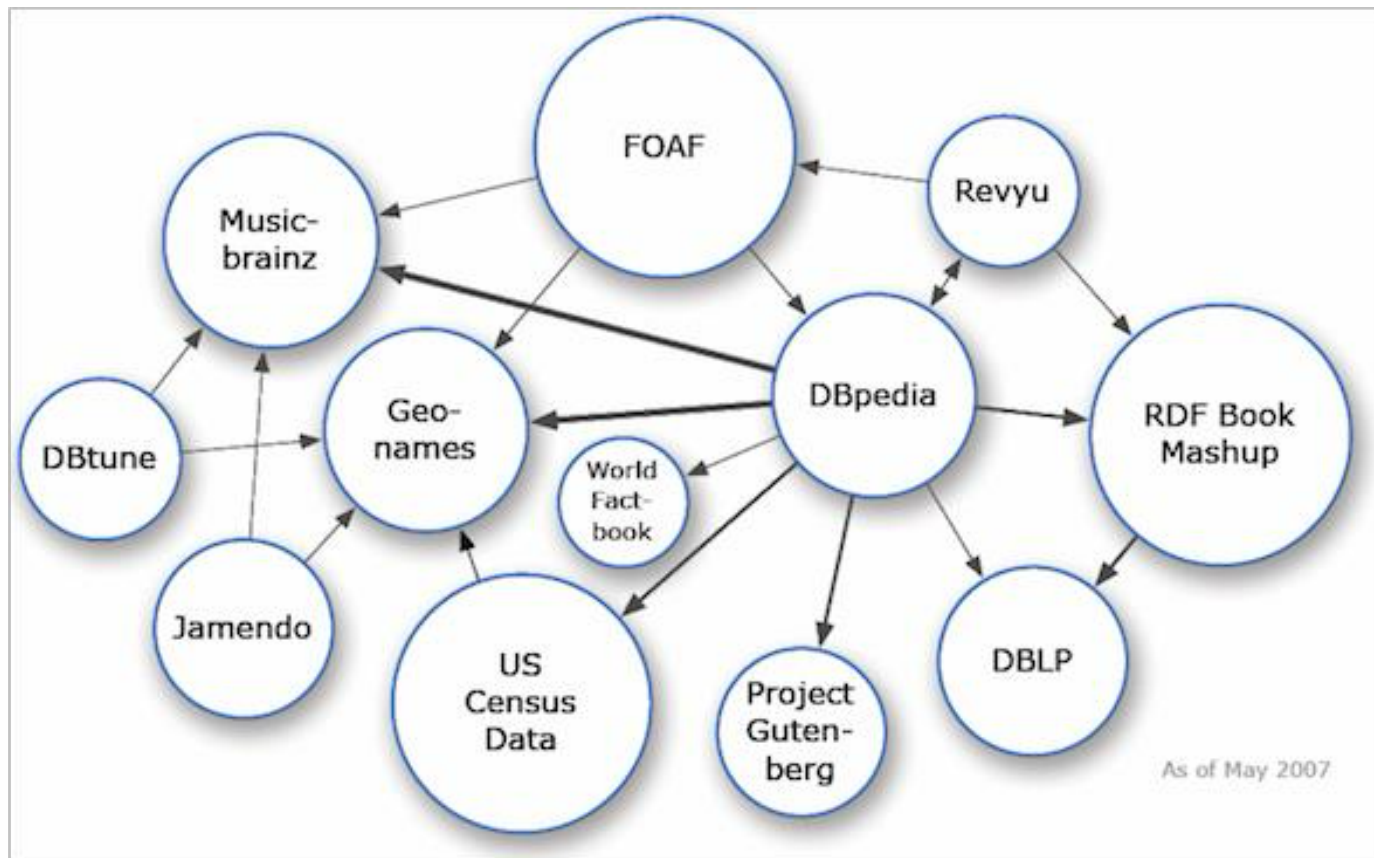


RDF Graph



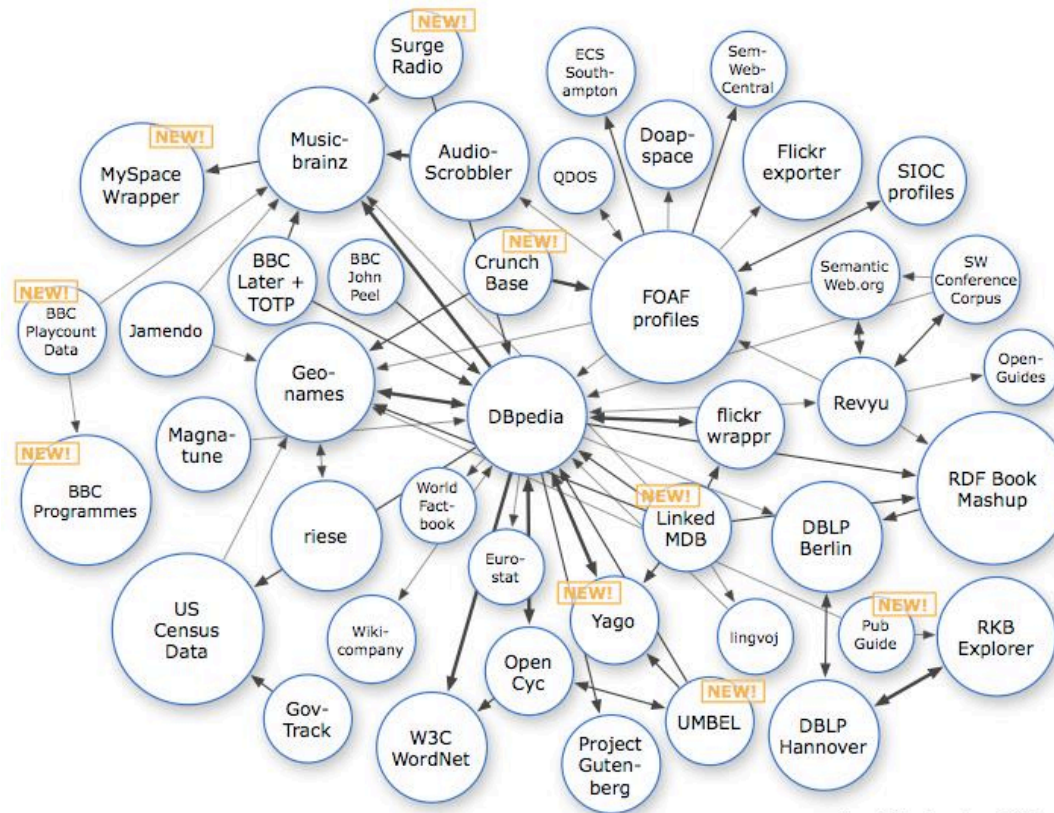
LOD Cloud → 2007

<https://lod-cloud.net/>



LOD Cloud → 2008

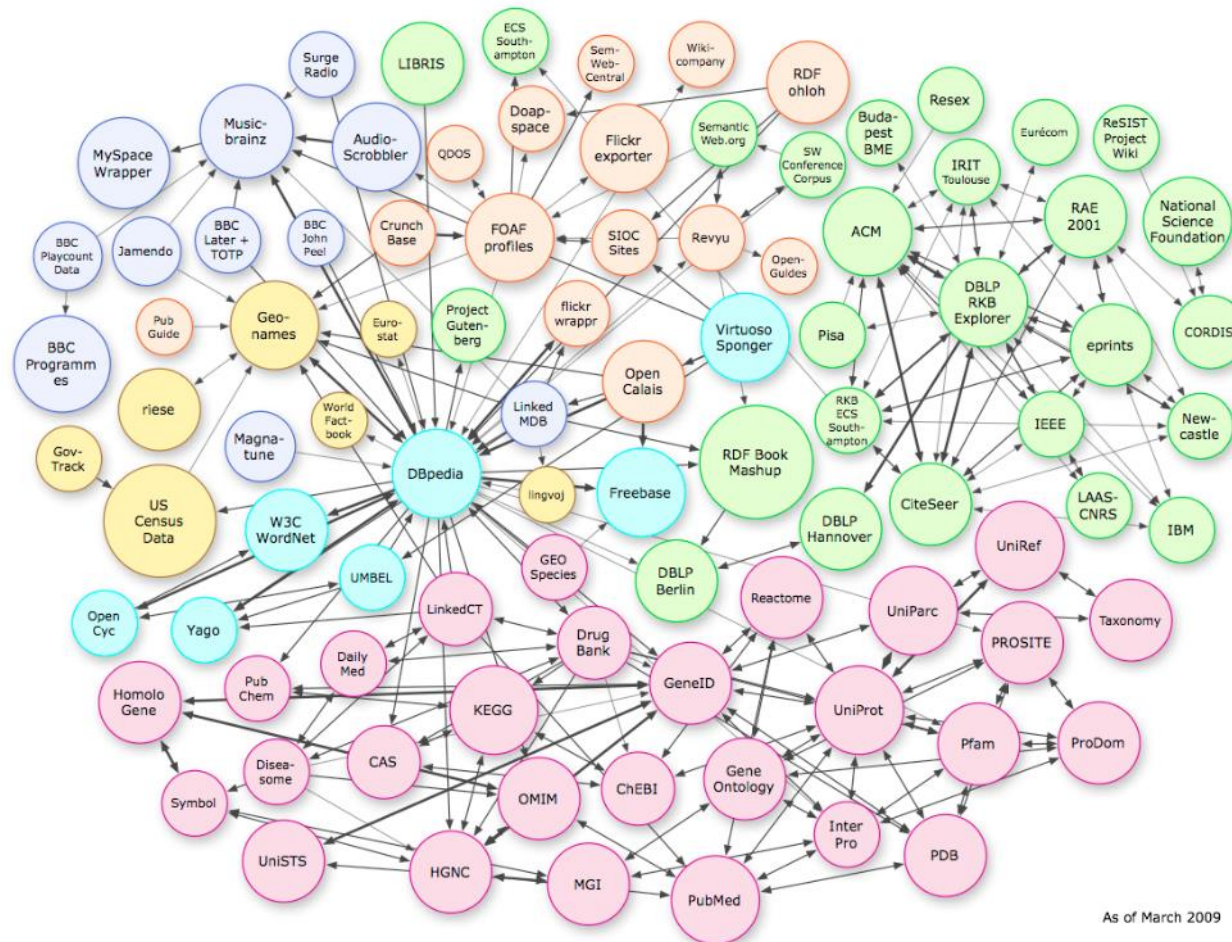
<https://lod-cloud.net/>



As of September 2008

LOD Cloud → 2009

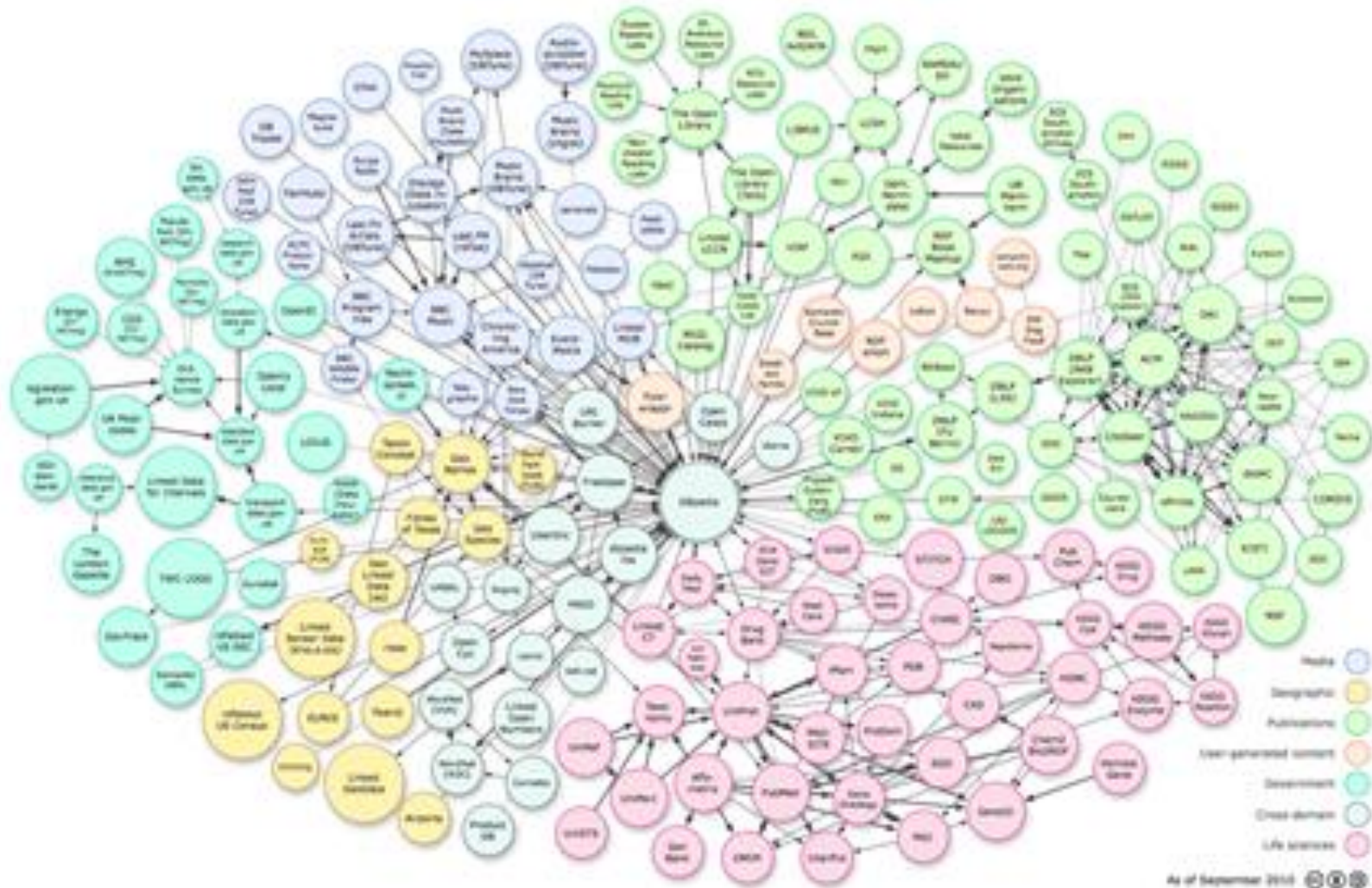
<https://lod-cloud.net/>



As of March 2009

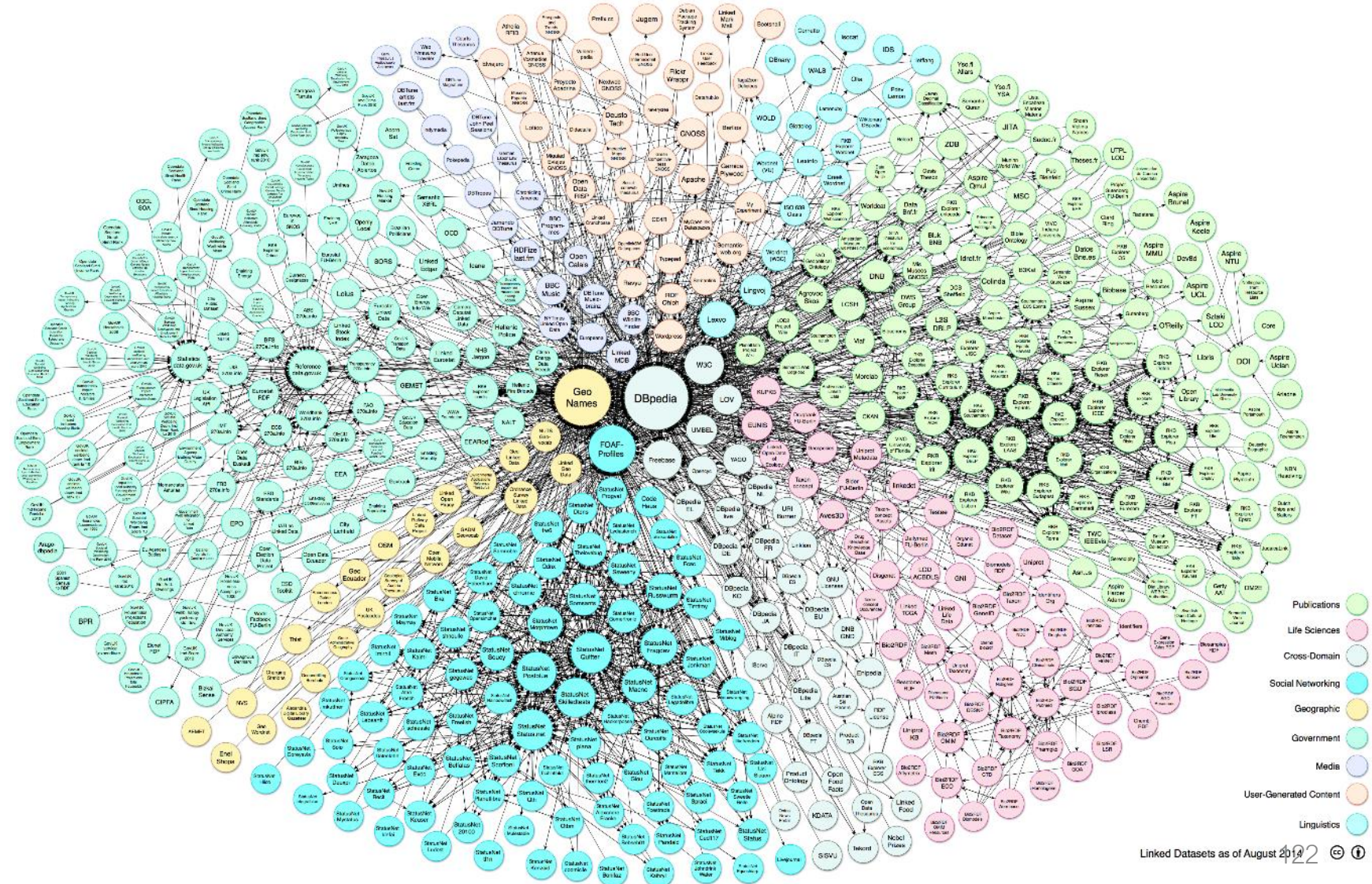
LOD Cloud → 2010

<https://lod-cloud.net/>



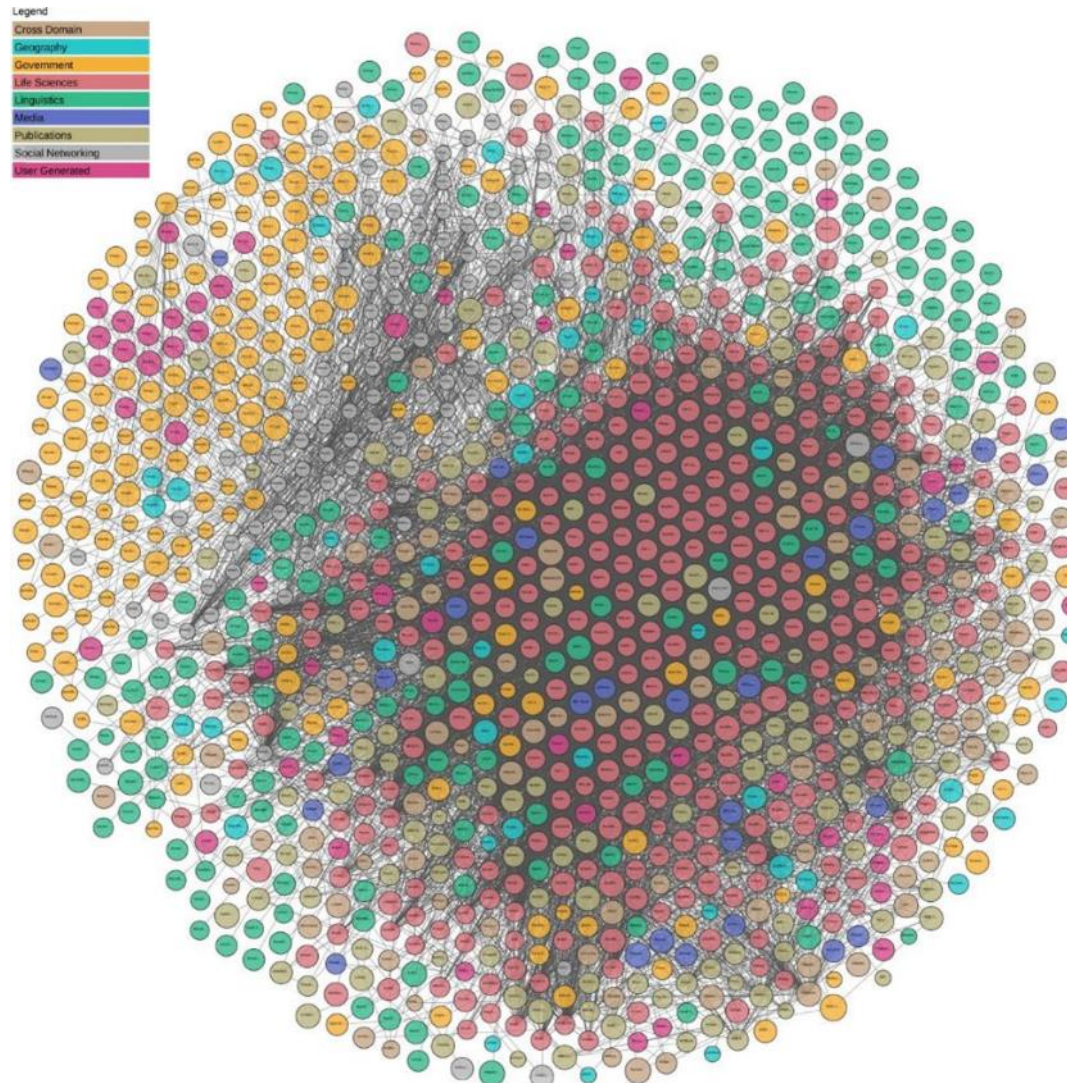
LOD Cloud → 2014

<https://lod-cloud.net/>



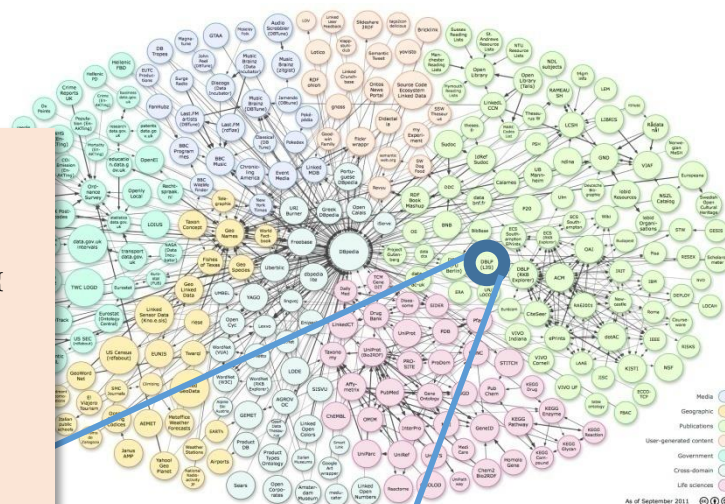
LOD Cloud → 2018

<https://lod-cloud.net/>



Process of accessing to other SPARQL end-point

```
PREFIX dcterms: <http://purl.org/dc/terms/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX swrc: <http://swrc.ontoware.org/ontology#>
SELECT DISTINCT ?s ?title ?name ?homepage ?page ?year WHERE {
    ?s rdfs:label ?title.
    FILTER regex(?title,".$kw.",'\i\').
    OPTIONAL { ?s dc:creator ?c_name.
                ?c_name rdfs:label ?name.
                ?s foaf:homepage ?homepage.
                ?s swrc:pages ?page.
                ?s dcterms:issued ?year. } }
order by desc(?year)
```



Linked OpenScholar Site



sudathip
sudathip's web site
(email)

BIO PROFILE PUBLICATIONS RESEARCHPROJECTS

Sudathip Chantorn
Lecturer
Department of Biotechnology Faculty of Science and Technology Thammasat University Rangsit Campus
p: 025644444 ext 2467 email (Website)
Professional Title/Affiliation: Lecturer

ตำแหน่ง: อาจารย์

Fullname: Sudathip Chantorn

ชื่อ-สกุล: สุดาทิพย์ ชันทน

ResearchArea: Biochemistry



XMLHttpRequest

responseXML



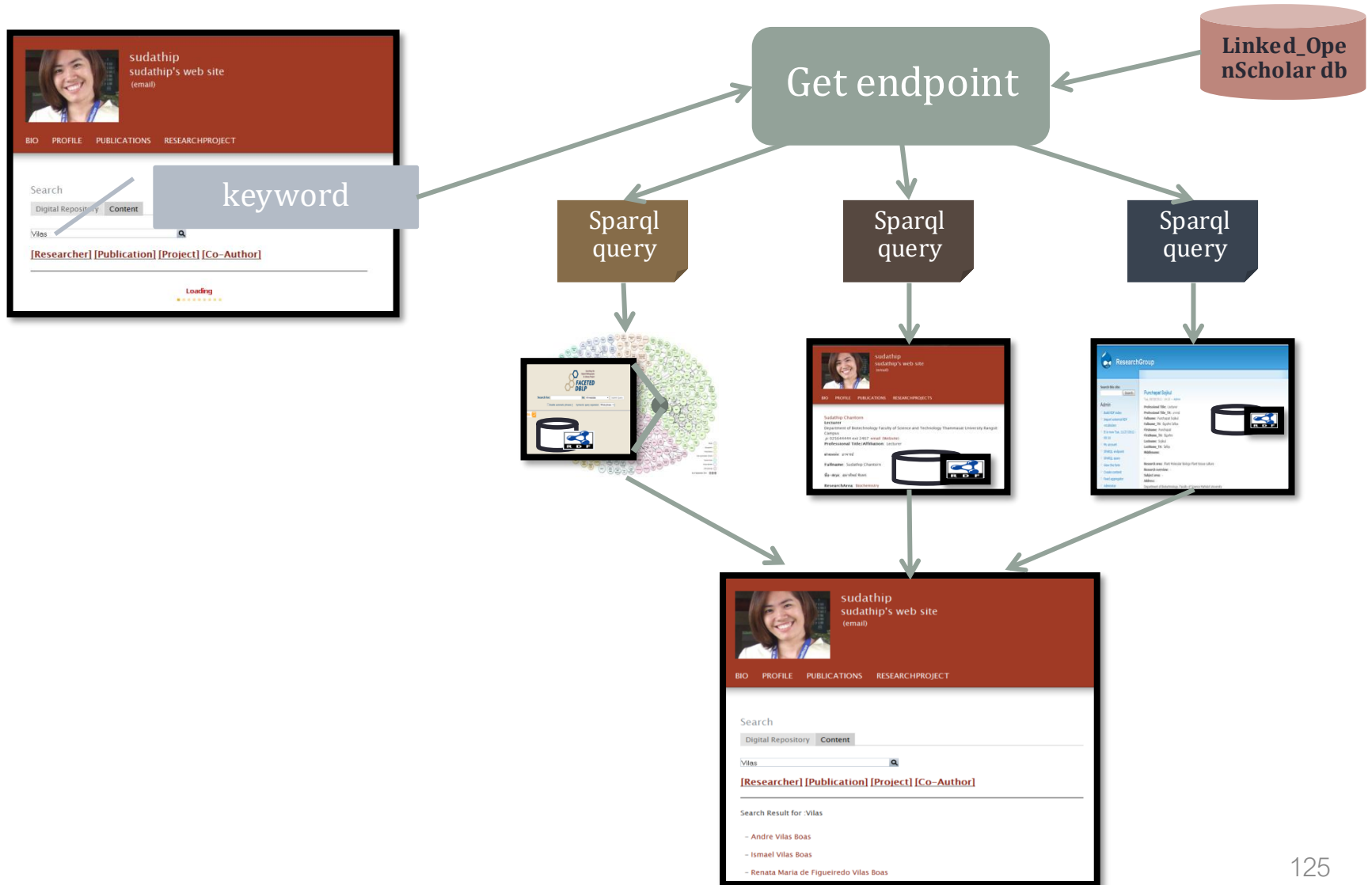
SPARQL Endpoint

Searching the Digital Bibliography & Library Project

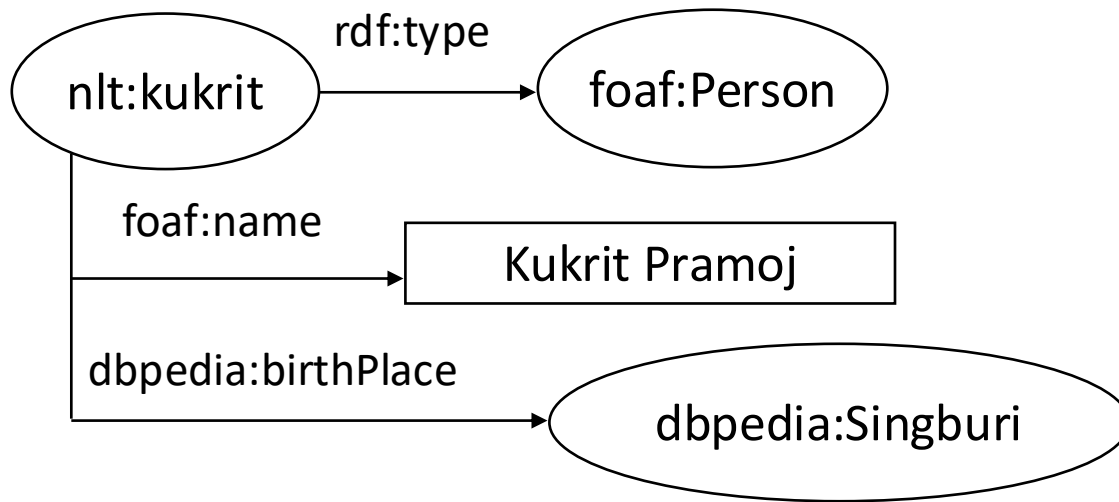
Submit Query

Whole phrase 2

Federated search



Linked Open Data



Prefixes

nlt: <<http://www.nlt.go.th/person/>>

rdf: <<http://www.w3.org/1999/02/22-rdf-syntax-ns#>>

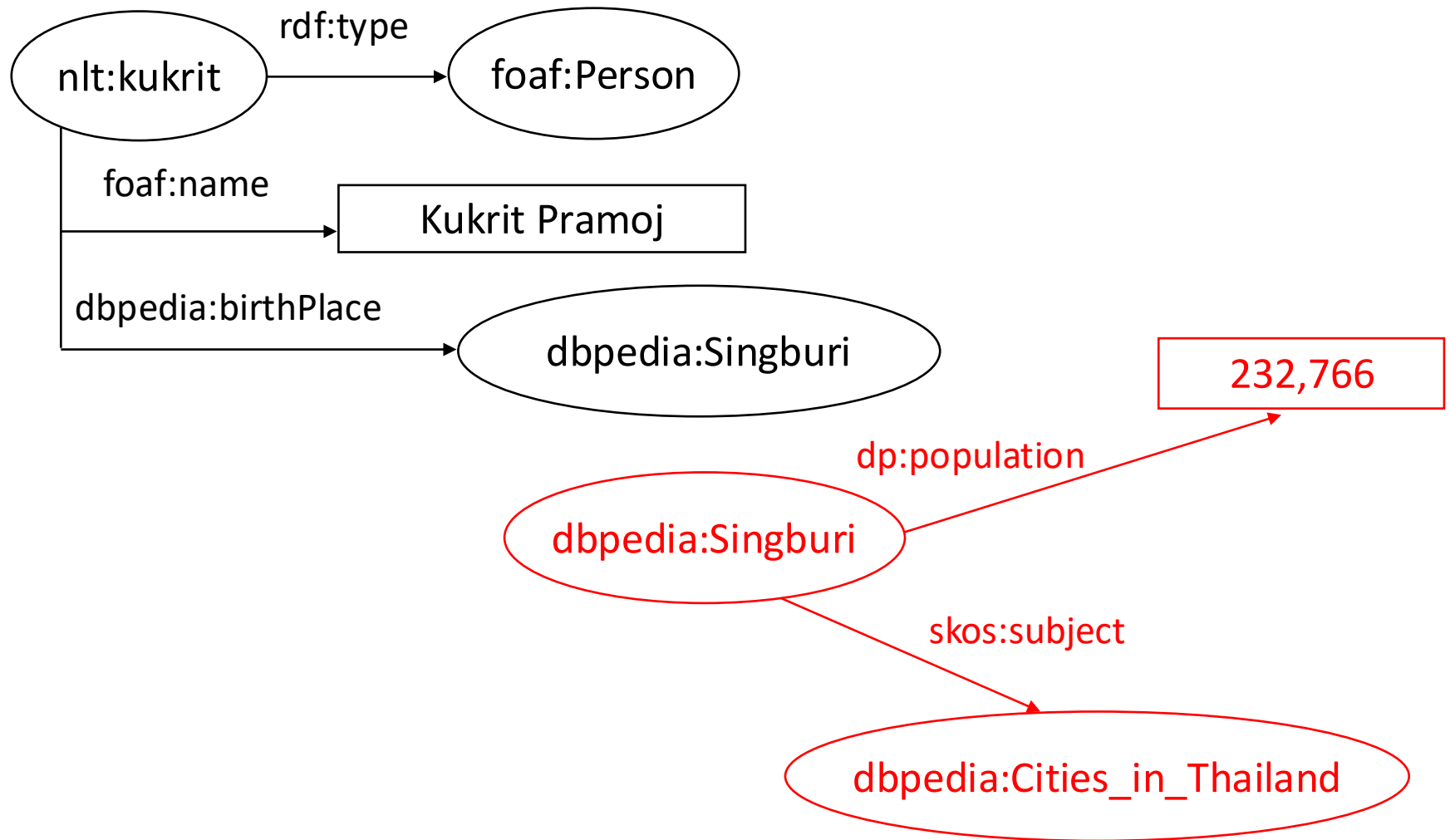
foaf: <<http://xmlns.com/foaf/0.1/>>

dbpedia: <<http://dbpedia.org/resource/>>

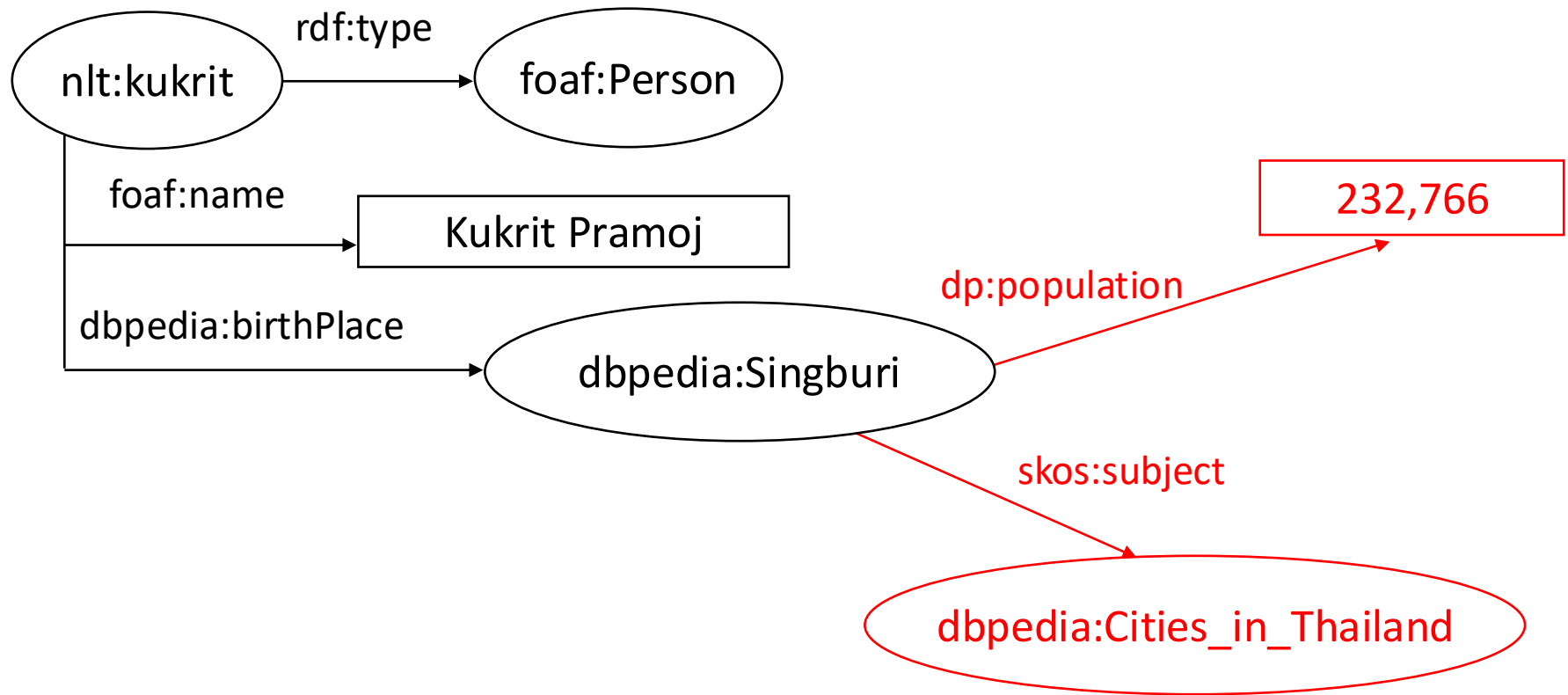
dp: <<http://dbpedia.org/property/>>

skos: <<http://www.w3.org/2004/02/skos/core#Collection>>

Linked Open Data



Linked Open Data



Your Data?

LINKED DATA

- ★ On the web, open license
- ★ ★ Machine-readable data
- ★ ★ ★ Non-proprietary format
- ★ ★ ★ ★ RDF standards
- ★ ★ ★ ★ ★ Linked RDF

IS YOUR DATA 5 ★ ?

See Also ...

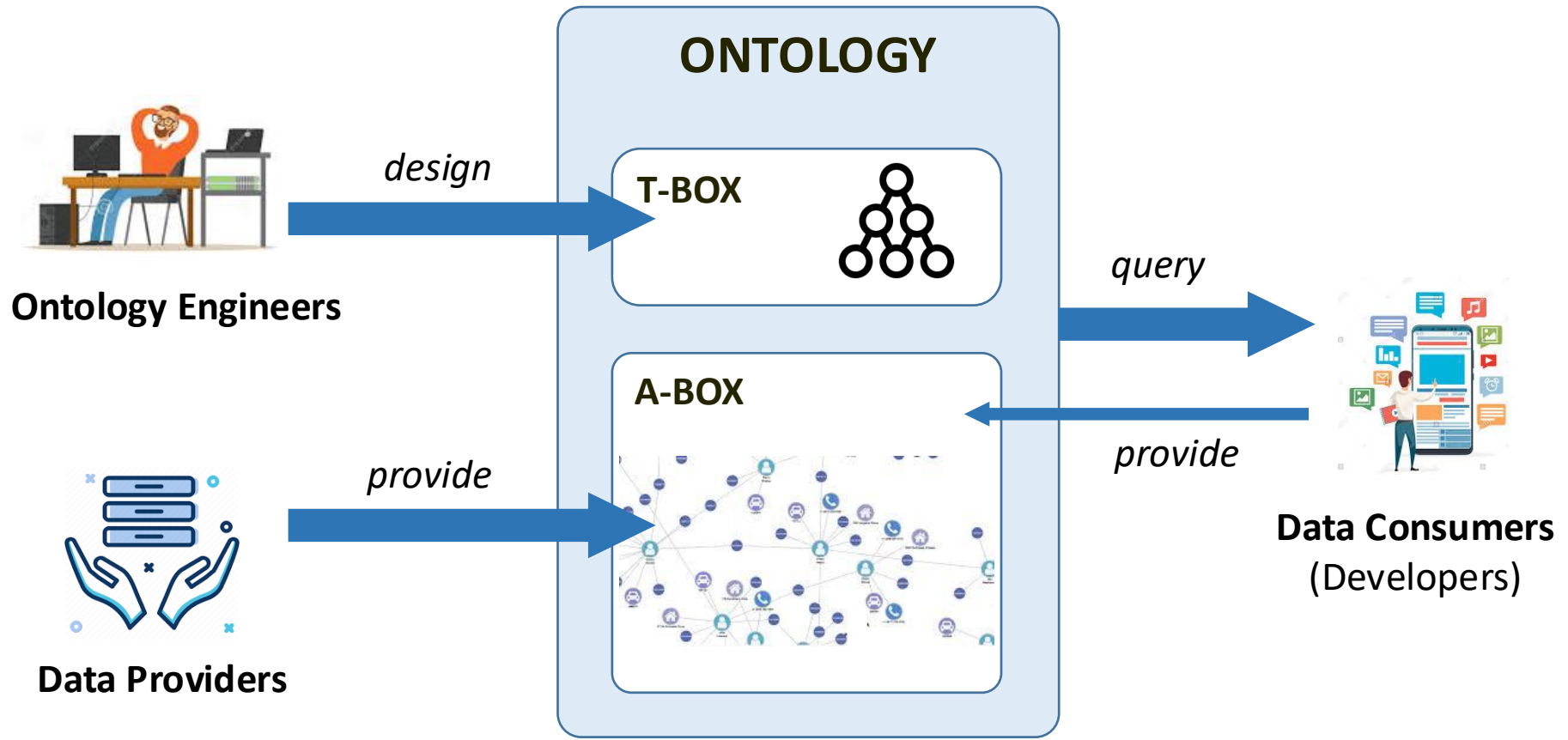
- **DBpedia**
 - <https://wiki.dbpedia.org/>
- **Linked Open Data**
 - <http://linkeddata.org/>
- **LOD Cloud**
 - <https://lod-cloud.net/>





Summary

Roles in LOD



Roles and Skills



Ontology Engineers

RDF

RDFS & OWL

Reasoning

SPARQL

LOD



Data Providers

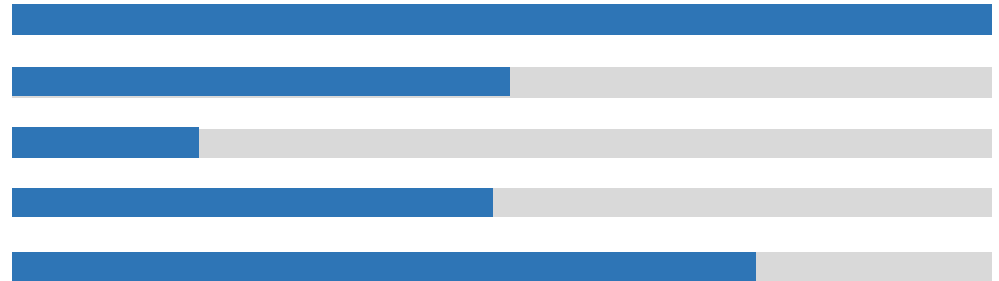
RDF

RDFS & OWL

Reasoning

SPARQL

LOD



Data Consumers (developers)

RDF

RDFS & OWL

Reasoning

SPARQL

LOD



Your Data?

LINKED DATA

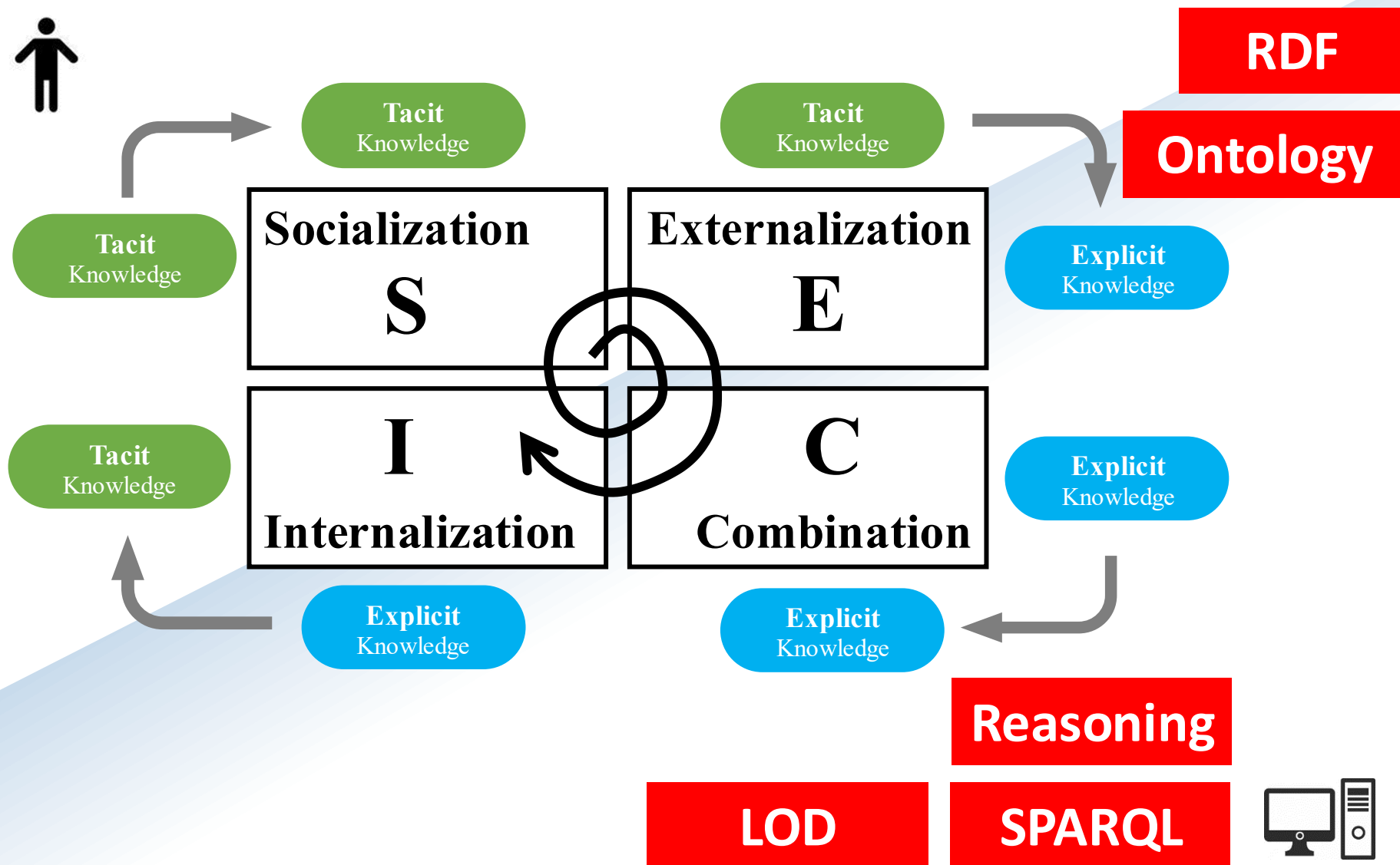
- ★ On the web, open license
- ★ ★ Machine-readable data
- ★ ★ ★ Non-proprietary format
- ★ ★ ★ ★ RDF standards
- ★ ★ ★ ★ ★ Linked RDF

IS YOUR DATA 5 ★ ?

Creating an Ontology

1. Get and Analyze Requirements
2. Create an Application Profiles
3. Create a Data Model with reused Vocabularies from Well-Known Ontologies
 - if no, create your own URIs and link them to well-known URIs.
4. Create an RDF graph following the model
 - Protégé (<https://protege.stanford.edu/>)
5. Create Semantic Web Rules (if any)
 - Protégé (<https://protege.stanford.edu/>)
6. Publish your Ontology via a SPARQL end-point
 - OpenLink Virtuoso (<https://virtuoso.openlinksw.com/>)
7. Maintain your ontology

Summary





ANY QUESTIONS ARE WELCOME



rathachai.creatier.pro

KMITL

